

DGOTECH · ECOSISTEMA DEPORTIVO

SportTrack · Frontend de Competencias

Pack de Entrega Documental

Repositorio: SportTrack-Front

Stack: React 18 · Vite 5 · React Router 6 · SignalR · Axios · Capacitor 7

Documentos incluidos: 7 · **Diagramas:** 10

Fecha de emisión: 27 de septiembre de 2026

Contenido

1. README

- Índice del pack
- Ficha del proyecto
- Propósito y propuesta de valor
- Alcance del pack
- Glosario
- Documentos históricos
- Pie del pack

2. 01 — Requerimientos de Usuario

01 — Requerimientos de Usuario

- Índice
- 1. Introducción y alcance
- 2. Actores y roles
- 3. Requerimientos Funcionales
- 4. Requerimientos No Funcionales (ISO 25010)
- 5. Historias de Usuario (Gherkin)
- 6. Reglas de Negocio
- 7. Matriz de trazabilidad
- Pie del pack

3. 02 — Casos de Uso

02 — Casos de Uso

- Índice
- 1. Introducción
- 2. Diagrama de casos de uso
- 3. Catálogo de casos de uso
- 4. Especificación de casos de uso
- Pie del pack

4. 03 — Diagramas de Flujo

03 — Diagramas de Flujo

- Índice
- 1. Introducción
 - (a) Alta/edición de evento y configuración de pruebas
 - (b) Siembra/sorteo y generación de fases
 - (c) Progresión ICF
 - (d) Ciclo de vida de una regata
 - (e) Cronometraje con resiliencia offline
 - (f) Autenticación/autorización por rol y plan
 - (g) Flujo maratón
 - (h) Auditoría de incidencias/errores
- Pie del pack

5. 04 — Manual de Usuario

04 — Manual de Usuario

- Índice
- 1. Introducción
- 2. Acceso y navegación
- 3. Público general
- 4. Club
- 5. Administrador de Federación / Admin
- 6. Largador
- 7. Cronometrista / Finalizador

- 8. Juez de Control
- 9. Control Técnico
- 10. SuperAdmin / Soporte
- 11. FAQ y resolución de problemas
- Pie del pack

6. 05 — Manual Técnico

05 — Manual Técnico

- Índice
- 1. Introducción y alcance
- 2. Arquitectura general
- 3. Stack y versiones reales
- 4. Estructura del código
- 5. Módulos y servicios
- 6. Contratos de API consumidos
- 7. SignalR: métodos y eventos
- 8. Estado y almacenamiento local
- 9. Planes SaaS y guards
- 10. Estrategia offline-first
- 11. Seguridad
- 12. Build y entornos
- 13. Despliegue
- 14. Mantenimiento y troubleshooting
- 15. Deuda técnica
- Pie del pack

7. 06 — Manual de Instalación y Despliegue

06 — Manual de Instalación y Despliegue

- Índice
- 1. Requisitos previos
- 2. Obtención del código
- 3. Instalación de dependencias
- 4. Configuración del entorno
- 5. Ejecución local
- 6. Build de producción
- 7. Despliegue web (Vercel)
- 8. Build Android con Capacitor
- 9. Verificación y checklist de entrega
- 10. Resolución de problemas de instalación
- Pie del pack

Pack de Entrega — Frontend SportTrack

Última actualización: 2026-09-27

Portada del paquete documental de entrega del **frontend** de gestión de competencias de remo y canotaje **SportTrack**. Este pack describe, en calidad de entrega formal, los requerimientos, casos de uso, flujos, manuales de usuario y la documentación técnica y de despliegue de la SPA React que consume la API unificada SportTrack / SIGDEF.

Nota de alcance. Este pack documenta **exclusivamente el frontend SportTrack-Front** . La API, la base de datos y el frontend federativo SIGDEF tienen sus propios packs de entrega en sus respectivos repositorios.

 **PDF consolidado del pack:** *Pack-Entrega-SportTrack-Front.pdf* Contiene los 7 documentos en orden, con todas las tablas y los **10 diagramas renderizados**. A4, 67 páginas, con índice navegable y marcadores (bookmarks) por sección.

Índice del pack

#	Documento	Contenido
—	README.md (este archivo)	Portada, ficha del proyecto, alcance, glosario y documentos históricos
1	<i>01 — Requerimientos de Usuario</i>	Actores, RF, RNF (ISO 25010), Historias de Usuario (Gherkin), Reglas de Negocio y matriz de trazabilidad
2	<i>02 — Casos de Uso</i>	Diagrama UML de casos de uso y especificación detallada de CU-01...CU-12
3	<i>03 — Diagramas de Flujo</i>	Diagramas Mermaid: alta de evento, siembra/fases, progresión ICF, ciclo de vida de regata, cronometraje offline, auth/plan, maratón y auditoría
4	<i>04 — Manual de Usuario</i>	Guía paso a paso por rol, tablas de campos, notas y FAQ / troubleshooting
5	<i>05 — Manual Técnico</i>	Arquitectura, stack real, estructura del código, contratos de API, SignalR, offline-first, seguridad, build y deuda técnica
6	<i>06 — Manual de Instalación y Despliegue</i>	Requisitos, instalación, entorno, ejecución local, build, Vercel y Android/Capacitor

Ficha del proyecto

Atributo	Valor
Nombre	SportTrack — Frontend de Competencias
Tipo	SPA (Single Page Application) web + app Android (Capacitor)
Dominio	Gestión y cronometraje de competencias de remo/canotaje (sprint y maratón)
Versión	1.0.0 (package.json)
Fecha del documento	2026-09-27
Repositorio	SportTrack-Front (c:\Users\EZ EQU\source\reposFront\SportTrack-Front)
Framework	React ^18.3.1 + React DOM ^18.3.1
Ruteo	react-router-dom ^6.26.2
Bundler	Vite ^5.4.10
Tiempo real	@microsoft/signalr ^8.0.7 (hub /hubs/timing)
HTTP	axios ^1.7.7
Reportes	jspdf ^4.2.1 + jspdf-autotable ^5.0.7 (PDF) · exportación CSV propia
Mapas	react-simple-maps ^3.0.0 , d3-geo ^3.1.1 , topojson-client ^3.1.0
UI	lucide-react ^1.8.0 , CSS propio
Móvil	@capacitor/core + @capacitor/android + @capacitor/cli 7.6.8
Backend consumido	API unificada SportTrack / SIGDEF (ASP.NET Core, .NET 10) con PostgreSQL
API (producción)	https://sporttrack-sigdef.onrender.com/api
Hub (producción)	https://sporttrack-sigdef.onrender.com/hubs/timing

Atributo	Valor
Deploy web	Vercel
Deploy móvil	Android (APK vía Capacitor)

Repositorios relacionados

Repo	Rol	Deploy
SportTrack-Front (este)	Frontend de competencias: eventos, inscripciones, fases, jueces, cronometraje, live, SuperAdmin SaaS	Vercel
SportTrack-Sigdef	Backend unificado (regatas + SIGDEF + SaaS), API REST + SignalR, PostgreSQL	Render
FrontSigdef	Frontend de administración federativa: atletas, clubes, delegados, tutores, SaaS	Vercel
WebSPA-SIGDEF	SPA pública/administrativa SIGDEF	Vercel

Propósito y propuesta de valor

SportTrack permite operar una competencia completa de punta a punta:

- **Planificación:** alta de eventos, pruebas por categoría/bote/distancia/sexo, inscripciones de clubes y cronograma.
- **Siembra y progresión:** sorteo de carriles, generación automática de fases (eliminatorias → semifinales → finales) y promoción de etapas conforme al sistema de progresión ICF.
- **Operación en cancha:** consola de largador (starter), consola de cronometrista/finalizador, juez de control y control técnico, con **resiliencia offline-first**.
- **Difusión:** pizarra pública de resultados en vivo (/resultados/:id) vía SignalR, sin login.
- **Administración SaaS:** panel SuperAdmin multi-tenant (federaciones, clubes, planes, usuarios, auditoría, soporte).

Diferenciador central: resiliencia ante mala señal. El cronometraje usa **colas locales** (localStorage/sessionStorage) + **outbox en servidor** + **respaldo PDF**, de modo que un corte de red no pierde los tiempos registrados.

Alcance del pack

Dentro del alcance:

- Requerimientos funcionales y no funcionales del frontend web y móvil.
- Casos de uso y flujos de los módulos implementados y activos en `main` a la fecha.
- Manuales de usuario por rol, técnico y de instalación/despliegue.
- Trazabilidad requerimiento ↔ caso de uso ↔ módulo de código.

Fuera del alcance (documentado como referencia):

- Implementación interna de la API, esquema de base de datos y migraciones (ver repo backend).
- Módulos federativos SIGDEF (atletas federativos, delegados, tutores) — ver `FrontSigdef`.
- Funcionalidades marcadas explícitamente en los documentos como **FUERA DE ALCANCE ACTUAL** (pendientes o comentadas en el código).

Glosario

Término	Definición
SPA	Single Page Application; aplicación web de página única basada en React.
Evento	Competencia completa (ej. "Campeonato Argentino de Sprint 2026"). Agrupa pruebas y fases.
Prueba	Combinación categoría + bote + distancia + sexo (ej. K1 200 m Masculino).
EventoPrueba	Instancia concreta de una prueba dentro de un evento; unidad sobre la que se generan fases.
Fase / Etapa	Instancia de competencia de un EventoPrueba (Eliminatorias, Semifinales, Finales).
Serie / Heat	Carrera individual dentro de una etapa.
Siembra / Sorteo	Asignación de carriles a inscripciones dentro de una serie.
Progresión ICF	Reglas de la Federación Internacional de Canotaje para avanzar atletas entre etapas (series → semis → finales, incluyendo repescas / best losers).

Término	Definición
Regata	Carrera en operación (una fase en curso). Sobre ella se emiten largada, tiempos y resultados.
t0 / Largada	Instante oficial de partida sincronizado con el reloj del servidor.
Outbox	Cola de tiempos pendientes persistida en el servidor para su commit posterior por Soporte.
Consola de juez	Interfaz operativa de cancha: largador, cronometrista/finalizador, juez de control, control técnico.
Finalizador	Etiqueta de UI para el rol Cronometrista (toma de tiempos de llegada).
Pizarra / Live	Vista pública <code>/resultados/:id</code> con resultados en vivo vía SignalR.
Plan SaaS	Nivel comercial (SIGDEF, SportTrack, Pack Dúo × tiers S/M/L) que habilita features por flags.
Flag de plan	Booleano derivado del plan: <code>accesoSigdef</code> , <code>accesoSportTrack</code> , <code>accesoControlesLive</code> , <code>accesoDashboardClub</code> , <code>permitirCargaImagenes</code> , <code>resultadosTiempoReal</code> , <code>exportacionPdf</code> , <code>maxAtletas</code> .
Guard	Componente de ruteo que protege acceso por rol (<code>ProtectedRoute</code>) o por plan (<code>PlanGuard</code>).
soporte_tecnico	Alias especial de usuario tratado como SuperAdmin por <code>authHelpers</code> .
X-Client-App	Header <code>sporttrack</code> inyectado por <code>api.js</code> para identificar el cliente ante la API.

Documentos históricos

Los siguientes documentos **se conservan** pero fueron redactados para el **backend** (dicen "Pack de Entrega: Backend SportTrack-v1" y referencian .NET). Se listan como material histórico y **no describen este frontend**:

Documento histórico	Motivo de la marca histórica
<code>ENTREGA_README.md</code>	Portada orientada al backend.
<code>PRESENTACION_TECNOLOGICA.md</code>	Presentación del stack del backend.
<code>Analisis/ANALISIS_TECNICO.md</code>	Análisis de arquitectura de la API.
<code>Diagramas/ARQUITECTURA_Y_DIAGRAMAS.md</code>	Diagramas del sistema server-side.
<code>Manuales/MANUAL_DE_USO.md</code>	Manual orientado a API/backend.
<code>Manuales/MANUAL_INSTALACION.md</code>	Instalación del backend (.NET).
<code>Manuales/GUIA_CRONOMETRISTA_MALA_SENAL.md</code>	Guía operativa (versión previa).
<code>Manuales/GUIA_SOPORTE_TIEMPOS_PENDIENTES.md</code>	Guía de soporte (versión previa).

Resto de la documentación del repositorio

Carpeta	Contenido
<code>../README.md</code>	Índice raíz de <code>docs/</code> .
<code>../guias-usuario/</code>	Manuales de uso, instalación, maratón y módulos de control.
<code>../casos-de-uso/</code>	Casos de uso generales.
<code>../criterios/</code>	Requerimientos y criterios.
<code>../tecnico/</code>	Arquitectura, SaaS, backup, admin DB, diagramas y modalidad maratón.
<code>../referencia/</code>	Material histórico y de referencia.

 **Corrección de inconsistencias conocidas.** La documentación previa referencia el backend como **.NET 8** (real: **.NET 10**), menciona **SQL Server** (real: **PostgreSQL**) y describe la autenticación como "solo cookie HttpOnly" (real: esquema **híbrido** Bearer en `localStorage` + cookie `X-Access-Token`). El presente pack refleja el estado real verificado contra el código.

Pie del pack

Documento anterior	Documento actual	Documento siguiente
—	README.md	01 — <i>Requerimientos de Usuario</i> →

01 — Requerimientos de Usuario

Proyecto: SportTrack — Frontend de Competencias **Última actualización:** 2026-09-27 **Norma de referencia:** ISO/IEC/IEEE 29148 (ingeniería de requerimientos) · ISO/IEC 25010 (calidad de producto)

Índice

- 1. Introducción y alcance
- 2. Actores y roles
- 3. Requerimientos Funcionales
- 4. Requerimientos No Funcionales (ISO 25010)
- 5. Historias de Usuario (Gherkin)
- 6. Reglas de Negocio
- 7. Matriz de trazabilidad
- 8. Pie del pack

1. Introducción y alcance

1.1 Objetivo del documento

Definir los requerimientos **funcionales y no funcionales** del frontend **SportTrack**, verificado contra el código fuente del repositorio `SportTrack-Front`, de modo que sirva como base contractual de la entrega y como referencia de trazabilidad hacia casos de uso, diagramas y módulos de implementación.

1.2 Alcance del sistema

El sistema cubre el ciclo completo de una competencia de remo/canotaje:

- 1. **Preparación:** alta de eventos, pruebas y cronograma; inscripciones de clubes.
- 2. **Siembra y armado:** sorteo de carriles y generación de fases con progresión ICF.
- 3. **Operación en vivo:** largada, cronometraje, validación de llegada, oficialización de resultados.
- 4. **Difusión:** pizarra pública de resultados en tiempo real y exportación de reportes PDF/CSV.
- 5. **Administración:** gestión SaaS multi-tenant, auditoría, mensajería y soporte.

1.3 Alcance de este documento

- **Incluye:** requerimientos del frontend web (Vercel) y de la app Android (Capacitor).
- **Excluye:** la implementación del backend (repo `SportTrack-Sigdef`) y los módulos federativos SIGDEF (repo `FrontSigdef`).
- Todo requerimiento no verificable en el código fue **omitido**. Lo que existe pero está desactivado se marca como **FUERA DE ALCANCE ACTUAL**.

1.4 Convenciones de identificadores

Prefijo	Significado	Ejemplo
RF-##	Requerimiento Funcional	RF-05
RNF-##	Requerimiento No Funcional	RNF-03
HU-##	Historia de Usuario	HU-07
RN-##	Regla de Negocio	RN-02
CU-##	Caso de Uso (documento 02)	CU-04

2. Actores y roles

2.1 Actores del sistema

Actor	Autenticación	Descripción
Público general	No requiere	Consulta resultados en vivo en /resultados/:id , home y planes.
Club	Requerida (rol Club)	Gestiona sus atletas, inscripciones, pagos y consulta sus eventos.
Administrador de Federación	Requerida (rol Admin)	Administra eventos, pruebas, clubes, usuarios y operación completa.
SuperAdmin	Requerida (rol SuperAdmin)	Administración SaaS global: federaciones, planes, backups, auditoría y soporte.
Largador (Starter)	Requerida (rol Largador)	Lanza las regatas desde la consola de largada.
Cronometrista / Finalizador	Requerida (rol Cronometrista)	Registra tiempos de llegada; la UI lo rotula "Finalizador".
Juez de Control	Requerida (rol JuezControl)	Valida y oficializa resultados de las regatas.
Control Técnico	Requerida (rol ControlTecnico)	Supervisiona técnica y operativamente; accede a varias consolas.

2.2 Roles técnicos y mapeo de código

Rol (UI)	Valor de rol en API	Ruta de aterrizaje (getDashboardPathForRole)	Guard
SuperAdmin	SuperAdmin	/super	requiredRole={['Admin', 'SuperAdmin']}
Admin (federación)	Admin	/super	requiredRole={['Admin', 'SuperAdmin']}
Club	Club	/club	requiredRole="Club"
Control Técnico	ControlTecnico	/control-tecnico	requiredRole={['Admin', 'SuperAdmin', 'ControlTecnico']} + requiereControlesLive
Largador	Largador	/jueces/largador	requiredRole= ['Admin', 'SuperAdmin', 'Largador', 'ControlTecnico']} + requiereControlesLive
Cronometrista (UI: "Finalizador")	Cronometrista	/jueces/llegada	requiredRole= ['Admin', 'SuperAdmin', 'Cronometrista', 'ControlTecnico']} + requiereControlesLive
Juez de Control	JuezControl	/juez-control	requiredRole={['Admin', 'SuperAdmin', 'JuezControl']} + requiereControlesLive
Soporte técnico (alias)	usuario soporte_tecnico	Tratado como SuperAdmin	isSuperAdminUser

Nota. El rol Cronometrista se muestra en la interfaz como "Finalizador". El alias soporte_tecnico no es un rol de la API sino un **usuario especial** que authHelpers.isSuperAdminUser reconoce como superadministrador.

3. Requerimientos Funcionales

3.1 Gestión de eventos

ID	Requerimiento	Prioridad
RF-01	El sistema debe permitir crear un evento con nombre, fechas, ubicación, federación y estado de inscripciones.	Alta
RF-02	El sistema debe permitir editar un evento existente, incluida la fecha del evento .	Alta
RF-03	El sistema debe permitir configurar las pruebas de un evento (categoría, bote, distancia, sexo), incluyendo eventos con múltiples categorías .	Alta
RF-04	El sistema debe permitir editar los horarios de las pruebas dentro del cronograma.	Alta

ID	Requerimiento	Prioridad
RF-05	El sistema debe permitir consultar los próximos eventos desde la home pública.	Media
RF-06	El sistema debe permitir marcar un evento como "todo manual" (sin consolas de juez), habilitando la carga manual de tiempos.	Media

3.2 Inscripciones

ID	Requerimiento	Prioridad
RF-07	El sistema debe permitir inscribir atletas/botes a una prueba de un evento, con validación de elegibilidad.	Alta
RF-08	El sistema debe permitir a un Club inscribir a sus propios atletas.	Alta
RF-09	El sistema debe soportar botes de equipo con tripulantes (K2, K4, C2, etc.).	Alta
RF-10	El sistema debe permitir consultar las inscripciones por evento/prueba.	Media
RF-11	El sistema debe integrar el registro de pago por inscripción.	Media

3.3 Fases y progresión ICF

ID	Requerimiento	Prioridad
RF-12	El sistema debe permitir realizar la siembra/sorteo y asignar carriles a las inscripciones.	Alta
RF-13	El sistema debe generar automáticamente las fases (series/etapas) de un EventoPrueba.	Alta
RF-14	El sistema debe permitir la generación manual de fases por colocaciones indicadas por el operador.	Media
RF-15	El sistema debe permitir promover atletas a la etapa siguiente conforme al sistema de progresión ICF.	Alta
RF-16	El sistema debe permitir editar la metadata de una fase (horario, carriles) y realizar batch update de fases.	Media
RF-17	El sistema debe permitir editar los detalles de una fase (<code>updateDetails</code>).	Media
RF-18	El sistema debe permitir eliminar una fase.	Baja
RF-19	El sistema debe exponer una auditoría de progresión por EventoPrueba.	Media

3.4 Cronometraje y operación de regata

ID	Requerimiento	Prioridad
RF-20	El sistema debe permitir al Largador iniciar una regata (t0) y solicitar su reinicio.	Alta
RF-21	El sistema debe permitir al Cronometrista/Finalizador capturar tiempos de llegada por carril.	Alta
RF-22	El sistema debe permitir al Juez de Control validar, enviar a revisión y oficializar resultados.	Alta
RF-23	El sistema debe permitir el control técnico de la operación.	Media
RF-24	El sistema debe permitir la carga manual de tiempos para planes sin controles live.	Media
RF-25	El sistema debe sincronizar el reloj con el servidor y calcular el t0 en el instante del click.	Alta
RF-26	El sistema debe emitir un timbre de largada audible en la consola del finalizador.	Baja
RF-27	El sistema debe permitir reiniciar una regata con motivo y categoría.	Alta

3.5 Resiliencia offline-first

ID	Requerimiento	Prioridad
RF-28	El sistema debe encolar localmente los tiempos capturados cuando no hay conexión.	Alta
RF-29	El sistema debe reintentar el envío de tiempos y largadas al restablecerse la conexión.	Alta
RF-30	El sistema debe persistir un outbox de tiempos en el servidor para su commit/discard.	Alta
RF-31	El sistema debe generar un respaldo PDF de los tiempos capturados.	Alta
RF-32	El sistema debe mantener la sesión extendida (token de 24 h) para turnos largos de cronometraje.	Alta

3.6 Resultados y pizarra en vivo

ID	Requerimiento	Prioridad
RF-33	El sistema debe mostrar una pizarra pública de resultados en vivo sin login en <code>/resultados/:id</code> .	Alta
RF-34	El sistema debe actualizar resultados en tiempo real vía SignalR.	Alta

ID	Requerimiento	Prioridad
RF-35	El sistema debe calcular posiciones por fase y diferencias de tiempo respecto del primero.	Alta
RF-36	El sistema debe permitir actualizar el estado de un resultado (provisional / oficial).	Alta

3.7 Modalidad maratón

ID	Requerimiento	Prioridad
RF-37	El sistema debe permitir configurar un evento en modalidad maratón (largada masiva por grupo).	Alta
RF-38	El sistema debe generar la largada de maratón con todos los inscriptos del grupo.	Alta
RF-39	El sistema debe permitir cargar y visualizar la start list / programa de maratón.	Alta
RF-40	El sistema debe agrupar resultados de maratón por clasificación.	Media

3.8 SaaS y federaciones

ID	Requerimiento	Prioridad
RF-41	El sistema debe permitir al SuperAdmin gestionar federaciones .	Alta
RF-42	El sistema debe permitir gestionar clubes , su estado activo/inactivo y su plan SaaS.	Alta
RF-43	El sistema debe permitir consultar métricas globales del SaaS.	Media
RF-44	El sistema debe permitir la gestión de logins/usuarios por rol y por plan.	Alta
RF-45	El sistema debe restringir el acceso a funcionalidades según los flags del plan contratado.	Alta

3.9 Auditoría y control de actividad

ID	Requerimiento	Prioridad
RF-46	El sistema debe registrar y listar la auditoría de errores de la aplicación.	Alta
RF-47	El sistema debe registrar y listar la auditoría de actividad agrupada por evento.	Alta
RF-48	El sistema debe encolar acciones de auditoría cuando no hay conexión y enviarlas luego.	Media
RF-49	El sistema debe permitir marcar un evento como "sin problemas" desde la auditoría.	Baja

3.10 Notificaciones y mensajería

ID	Requerimiento	Prioridad
RF-50	El sistema debe mostrar una campana de notificaciones segmentada por categoría (requiere acción, mensajes, novedades).	Media
RF-51	El sistema debe notificar nuevos eventos y mensajes en tiempo real.	Media
RF-52	El sistema debe permitir la mensajería privada por hilos entre usuarios y hacia clubes.	Media
RF-53	El sistema debe permitir el envío de comunicaciones masivas y campañas.	Media

3.11 Reportes y exportación

ID	Requerimiento	Prioridad
RF-54	El sistema debe exportar a PDF : fase, grupo, prueba, start list de maratón, cronograma completo, programa inicial, programa de maratón, cronograma de regatas y respaldo del cronometrista.	Alta
RF-55	El sistema debe exportar datos a CSV .	Media
RF-56	El sistema debe permitir descargar el PDF de respaldo desde la consola del finalizador.	Alta

3.12 Autenticación, permisos y plataforma

ID	Requerimiento	Prioridad
RF-57	El sistema debe autenticar por usuario/contraseña y validar la sesión en <code>/auth/me</code> .	Alta
RF-58	El sistema debe restaurar el plan del usuario desde el almacenamiento local al reanudar la sesión.	Alta

ID	Requerimiento	Prioridad
RF-59	El sistema debe permitir a la federación habilitar/deshabilitar operadores por rol en eventos.	Media
RF-60	El sistema debe ejecutarse como app Android mediante Capacitor, con permiso de red al backend.	Media

4. Requerimientos No Funcionales (ISO 25010)

4.1 Adecuación funcional

ID	Característica	Requerimiento
RNF-01	Compleitud funcional	Toda funcionalidad de operación de competencia (RF-01...RF-56) debe estar accesible desde la UI web sin pasos manuales sobre la base de datos.
RNF-02	Corrección funcional	Las posiciones y diferencias de tiempo deben calcularse con redondeo a centésimas a partir del <code>t0</code> sincronizado con el servidor.

4.2 Eficiencia de desempeño

ID	Característica	Requerimiento
RNF-03	Comportamiento temporal	La confirmación visual de un tiempo capturado debe ser inmediata (optimista) y no bloquear la captura del siguiente carril.
RNF-04	Utilización de recursos	El bundle debe construirse con Vite y dividirse para no descargar <code>jspdf</code> , mapas ni módulos de jueces en rutas que no los usan.
RNF-05	Capacidad	La pizarra en vivo debe soportar la actualización simultánea de múltiples series sin recarga de página.

4.3 Usabilidad

ID	Característica	Requerimiento
RNF-06	Reconocibilidad	La consola del finalizador debe exponer un botón grande por carril y una campana audible de largada.
RNF-07	Accesibilidad operativa	La operación de cancha debe ser usable en tablets y móviles (layout de jueces <code>JudgesLayout</code>).
RNF-08	Estética/consistencia	La UI debe respetar el tema (claro/oscuro) persistido en <code>localStorage</code> y feedback por toasts.

4.4 Fiabilidad

ID	Característica	Requerimiento
RNF-09	Tolerancia a fallos	Ante pérdida de conexión, los tiempos deben persistir localmente y reintentarse automáticamente (colas + outbox).
RNF-10	Recuperabilidad	El respaldo PDF del cronometrista debe permitir reconstruir manualmente los tiempos ante fallos prolongados.
RNF-11	Disponibilidad operativa	La app debe reanudar la conexión SignalR automáticamente y re-suscribirse a los grupos de evento y carrera.

4.5 Seguridad

ID	Característica	Requerimiento
RNF-12	Autenticidad	Cada request debe enviar el JWT en <code>Authorization: Bearer</code> y el header <code>X-Client-App: sporttrack</code> .
RNF-13	Confidencialidad	El acceso a consolas y paneles debe protegerse por rol (<code>ProtectedRoute</code>) y por plan (<code>PlanGuard</code>).
RNF-14	Integridad	El <code>t0</code> no debe regenerarse: si falla el canal SignalR, se usa HTTP o se encola, nunca se recalcula el instante de largada.
RNF-15	Trazabilidad de seguridad	Las acciones sensibles (reinicio de regata, cambios de estado) deben quedar registradas en auditoría.

4.6 Compatibilidad

ID	Característica	Requerimiento
RNF-16	Interoperabilidad	El frontend debe consumir la API REST y el hub SignalR de la API unificada sin acoplarse a versiones del backend.

ID	Característica	Requerimiento
RNF-17	Coexistencia	El frontend debe convivir con SIGDEF sobre la misma API/BD, diferenciándose por módulos y guards.
RNF-18	Portabilidad	El mismo código debe compilar como web (Vercel) y como app Android (<code>build:android</code>).

4.7 Mantenibilidad

ID	Característica	Requerimiento
RNF-19	Modularidad	La lógica de operación debe residir en servicios y hooks reutilizables (p. ej. <code>useResultados.js</code>).
RNF-20	Reusabilidad	Los vectores HTTP deben centralizarse en <code>ENDPOINTS</code> (<code>utils/constants.js</code>).
RNF-21	Analizabilidad	La app debe exponer auditoría de errores y de actividad consultable desde Soporte.
RNF-22	Modificabilidad	Los flags de plan y roles deben derivarse de helpers únicos (<code>planHelpers</code> , <code>authHelpers</code>).

5. Historias de Usuario (Gherkin)

Formato: Como `<rol>` quiero `<acción>` para `<beneficio>` . Criterios de aceptación en Gherkin (`Dado/Cuando/Entonces`).

HU-01 — Alta de evento con pruebas

Como Administrador de Federación **quiero** crear un evento y configurar sus pruebas **para** dejarlo listo para recibir inscripciones.

Escenario: Crear evento con una prueba
Dado que estoy autenticado como Admin
Y tengo acceso al sistema SportTrack según mi plan
Cuando completo el formulario de evento con nombre, fecha y ubicación
Y configuro una prueba (categoría, bote, distancia, sexo)
Y guardo el evento
Entonces el evento queda disponible en la grilla de eventos
Y aparece en el cronograma con su horario

Escenario: Editar la fecha de un evento existente
Dado un evento creado con fecha anterior
Cuando modifico la fecha del evento y guardo
Entonces el cronograma se recalcula con la nueva fecha

HU-02 — Inscripción de atleta por club

Como Club **quiero** inscribir a mis atletas en las pruebas **para** participar de la competencia.

Escenario: Inscripción válida
Dado que estoy autenticado como usuario de Club
Y el evento tiene inscripciones abiertas
Cuando inscribo un atleta en una prueba compatible
Entonces la inscripción queda registrada y visible en la lista
Y su elegibilidad fue validada por el sistema

Escenario: Inscripción rechazada por elegibilidad
Dado un atleta que no cumple la elegibilidad de la prueba
Cuando intento inscribirlo
Entonces el sistema rechaza la inscripción y muestra el motivo

HU-03 — Generación automática de fases

Como Administrador **quiero** generar las fases de una prueba automáticamente **para** obtener series, semifinales y finales sin armarlas a mano.

Escenario: Generar fases con progresión ICF

Dado un EventoPrueba con inscripciones confirmadas

Cuando ejecuto "Generar fases"

Entonces el sistema crea las etapas (Eliminatorias/Semifinales/Finales) con sus series

Y cada atleta aparece en la serie y carril sorteados

HU-04 — Promover etapa ICF

Como Administrador **quiero** promover los clasificados a la etapa siguiente **para** continuar la competencia conforme al reglamento ICF.

Escenario: Promover clasificados

Dado un EventoPrueba con Eliminatorias finalizadas y resultados cargados

Cuando ejecuto "Promover"

Entonces los clasificados por regla ICF se asignan a la etapa siguiente

Y los no clasificados quedan fuera de la progresión

Y queda registro en la auditoría de progresión

HU-05 — Largada de regata

Como Largador **quiero** dar la largada de una serie **para** iniciar oficialmente el cronometraje.

Escenario: Largada exitosa

Dado que estoy autenticado como Largador con controles live habilitados

Y la serie está en estado "lista"

Cuando presiono "Largar"

Entonces se registra el t0 sincronizado con el reloj del servidor

Y el cronometrista recibe la señal de largada con timbre

Escenario: Largada sin conexión

Dado que la conexión está caída

Cuando presiono "Largar"

Entonces el t0 se guarda localmente y se encola

Y se reintenta automáticamente al restablecerse la conexión

Y el t0 NO se regenera

HU-06 — Cronometraje de llegada

Como Cronometrista (Finalizador) **quiero** registrar el tiempo de llegada de cada carril **para** obtener los resultados de la serie.

Escenario: Captura de tiempos online

Dado que la regata fue largada y hay conexión

Cuando registro el tiempo de llegada de cada carril

Entonces el tiempo se envía al servidor y se refleja en la pizarra

Y se emite el evento TimeReceived a los suscriptores

Escenario: Captura de tiempos offline con respaldo

Dado que la conexión se pierde durante la serie

Cuando registro los tiempos de los carriles

Entonces los tiempos se guardan en la cola local

Y puedo descargar el PDF de respaldo

Y al volver la conexión los tiempos se sincronizan

HU-07 — Juez de Control oficializa resultados

Como Juez de Control **quiero** validar y oficializar los resultados **para** publicarlos como definitivos en la pizarra.

Escenario: Oficializar una serie

Dado que la serie tiene todos los tiempos cargados
 Cuando reviso los resultados y presiono "Oficializar"
 Entonces el estado de los resultados pasa a oficial
 Y la pizarra pública muestra los resultados oficiales
 Y se registra la acción en auditoría

HU-08 — Pizarra pública en vivo

Como público general **quiero** ver los resultados en vivo sin iniciar sesión **para** seguir la competencia desde mi dispositivo.

Escenario: Consulta pública

Dado que accedo al enlace /resultados/:id
 Cuando la competencia está en curso
 Entonces veo las series, los tiempos y las posiciones actualizadas en tiempo real
 Y no se me solicita inicio de sesión

HU-09 — Reiniciar regata

Como Largador **quiero** reiniciar una regata con un motivo **para** repetirla tras una mala largada o incidente.

Escenario: Reinicio por mala largada

Dado una regata en curso
 Cuando solicito el reinicio indicando categoría "Mala largada" y detalle
 Entonces el sistema notifica el evento RaceReset a los conectados
 Y la serie vuelve a estado listo para una nueva largada
 Y el motivo fuerza al menos un texto cuando la categoría es "Otro"

HU-10 — Configuración de maratón

Como Administrador **quiero** configurar un evento en modalidad maratón **para** operar largadas masivas por grupo.

Escenario: Generar largada de maratón

Dado un evento en modalidad maratón con inscripciones
 Cuando ejecuto la generación de largada de maratón
 Entonces se crea una fase "Largada" con todos los inscriptos del grupo
 Y el programa/start list queda disponible para exportar

HU-11 — Soporte de tiempos pendientes

Como Soporte técnico **quiero** ver y confirmar las colas temporales de tiempos **para** recuperar capturas que no llegaron al servidor.

Escenario: Confirmar cola temporal

Dado que existe una entrada pendiente en el outbox del servidor
 Cuando la selecciono y presiono "Confirmar"
 Entonces los tiempos se consolidan en la fase correspondiente
 Y la entrada desaparece de la cola

Escenario: Descartar cola temporal

Dado que existe una entrada pendiente inválida
 Cuando presiono "Descartar" y confirmo
 Entonces la cola se elimina del outbox

HU-12 — Restricción por plan SaaS

Como SuperAdmin **quiero** que cada plan habilite solo las features contratadas **para** respetar la comercialización del SaaS.

Escenario: Acceso bloqueado por plan

Dado un usuario con un plan que no incluye controles live

Cuando intenta entrar a /jueces/largador

Entonces el sistema muestra la pantalla "Función exclusiva del Ecosistema"

Y ofrece actualizar el plan o cerrar sesión

6. Reglas de Negocio

ID	Regla	Origen en código
RN-01	Los planes SaaS se identifican por ID: SIGDEF 1–3, SportTrack 4–6, Pack Dúo 7–9.	planHelpers.js
RN-02	Solo los planes tier L (IDs 6 y 9) habilitan accesoControlesLive.	planHelpers.js
RN-03	El flag maxAtletas = -1 significa "sin límite".	planHelpers.js
RN-04	La creación de logins de club requiere accesoDashboardClub; la de logins de juez requiere accesoControlesLive.	canCreateClubLogin, canCreateJudgeLogin
RN-05	SuperAdmin (y el alias soporte_tecnico) omiten las restricciones de plan.	PlanGuard, ProtectedRoute, authHelpers
RN-06	La ruta /jueces/carga-manual no exige requiereControlesLive; está disponible desde el plan Esencial.	App.jsx
RN-07	Un login de Club debe estar vinculado obligatoriamente a un club.	AuthService.register
RN-08	La sesión del cronometrista se extiende a 24 h para cubrir turnos completos.	tokenUtils, FinisherDashboard
RN-09	El t0 nunca se regenera: se entrega por SignalR → HTTP → cola persistente.	TimingSignalRService.deliverRaceStart
RN-10	El reinicio de regata requiere motivo; si la categoría es "Otro", el detalle debe tener al menos 10 caracteres.	reiniciarFaseConstants.isReiniciarMotivoValid
RN-11	El respaldo PDF del cronometrista tiene TTL de 24 h.	timingBackupService
RN-12	Los eventos "todo manual" habilitan handleGenerarManual / isManualTiming y ocultan controles de simulación.	useResultados.js, ConfigurarMaratonModal
RN-13	En modalidad maratón la largada es masiva: una única fase "Largada" con todos los inscriptos del grupo.	FaseService.generarLargadaMaraton
RN-14	Los archivos .env* no se versionan; las variables se cargan en Vercel → Settings.	.gitignore, .env.example
RN-15	La autenticación es híbrida: Bearer en localStorage + cookie X-Access-Token; en Capacitor solo Bearer (las cookies cross-origin rompen el login).	api.js, authHelpers.js
RN-16	Toda request debe enviar X-Client-App: sporttrack.	api.js
RN-17	Ante un 401, el sistema limpia USER_DATA y AUTH_TOKEN del almacenamiento.	api.js (interceptor de respuesta)

7. Matriz de trazabilidad

RF	Módulo / componente	HU relacionada	CU (doc 02)
RF-01, RF-02, RF-06	GestionEventosSection, EventForm	HU-01	CU-02
RF-03, RF-04	ConfigurarPruebasModal, PruebaForm, useResultados.handleUpdateFaseHorario	HU-01	CU-02

RF	Módulo / componente	HU relacionada	CU (doc 02)
RF-07, RF-08, RF-09, RF-10	RegistroInscripcionesSection, InscripcionAtletaModal, inscripcionEligibilityUtils	HU-02	CU-03
RF-12, RF-13, RF-16, RF-17, RF-18, RF-19	ProgressionEngine, FaseService, ProgressionAuditPage	HU-03	CU-04
RF-14	useResultados.handleGenerarManual	HU-03	CU-04
RF-15	FaseService.promover, promotionHelpers	HU-04	CU-05
RF-20, RF-25, RF-26, RF-27	StarterDashboard, raceStartBell, ReiniciarFaseDialog	HU-05, HU-09	CU-06, CU-08
RF-21, RF-28, RF-29, RF-31, RF-32, RF-56	FinisherDashboard, timingSubmitQueue, timingBackupService	HU-06	CU-07
RF-22	JuezControlDashboard, TimingSignalRService.updateResultStatus	HU-07	CU-07
RF-23, RF-24	ControlesSection, ManualTiming	HU-06	CU-07
RF-30	SoporteSection (colas temporales), SupportService	HU-11	CU-10
RF-33, RF-34, RF-35, RF-36	LiveResults, ResultadosTable, resultadosHelpers	HU-08	CU-09
RF-37, RF-38, RF-39, RF-40	ConfigurarMaratonModal, maratonScheduleUtils, maratonStartListUtils	HU-10	CU-11
RF-41, RF-42, RF-43, RF-44	GestionFederacionesSection, SaaSManagement, GestionLoginsSection	HU-12	CU-12
RF-45	PlanGuard, ProtectedRoute, planHelpers	HU-12	CU-12
RF-46, RF-47, RF-48, RF-49	SoporteSection, ActividadPorEventoPage, AuditoriaService, auditActionQueue	—	CU-12
RF-50, RF-51	NotificationCenter, notificationHelpers	—	CU-01
RF-52, RF-53	MessageService, MensajesSection, DestinatariosMultiSelect	—	CU-01
RF-54, RF-55	PdfExportService, CsvExportService	HU-08	CU-09
RF-57, RF-58, RF-59, RF-60	AuthContext, AuthService, GestionLoginsSection, Capacitor	HU-12	CU-01, CU-12
RNF-09, RNF-10, RNF-14	timingSubmitQueue, timingOutboxService, timingBackupService	HU-05, HU-06	CU-07, CU-08
RNF-12 a RNF-15	api.js, ProtectedRoute, PlanGuard	HU-12	CU-01, CU-12
RNF-16 a RNF-22	Arquitectura general de src/	—	—

Pie del pack

Documento anterior	Documento actual	Documento siguiente
← README	01 — Requerimientos de Usuario	02 — Casos de Uso →

[README](#) · [01](#) · [02](#) · [03](#) · [04](#) · [05](#) · [06](#)

02 — Casos de Uso

Proyecto: SportTrack — Frontend de Competencias **Última actualización:** 2026-09-27 **Notación:** UML (casos de uso) · Especificación de flujos según ISO/IEC/IEEE 29148

Índice

- 1. Introducción
- 2. Diagrama de casos de uso
- 3. Catálogo de casos de uso
- 4. Especificación de casos de uso
 - CU-01 — Iniciar sesión
 - CU-02 — Crear / editar evento y configurar pruebas
 - CU-03 — Inscribir atleta
 - CU-04 — Generar fases automáticamente
 - CU-05 — Promover etapa ICF
 - CU-06 — Cronometrar largada
 - CU-07 — Cronometrar llegada y validar/oficializar
 - CU-08 — Reiniciar regata
 - CU-09 — Consultar resultados en vivo y exportar PDF/CSV
 - CU-10 — Consola de soporte de colas de tiempos
 - CU-11 — Configurar maratón
 - CU-12 — Gestionar federaciones, planes y logins
- 5. Pie del pack

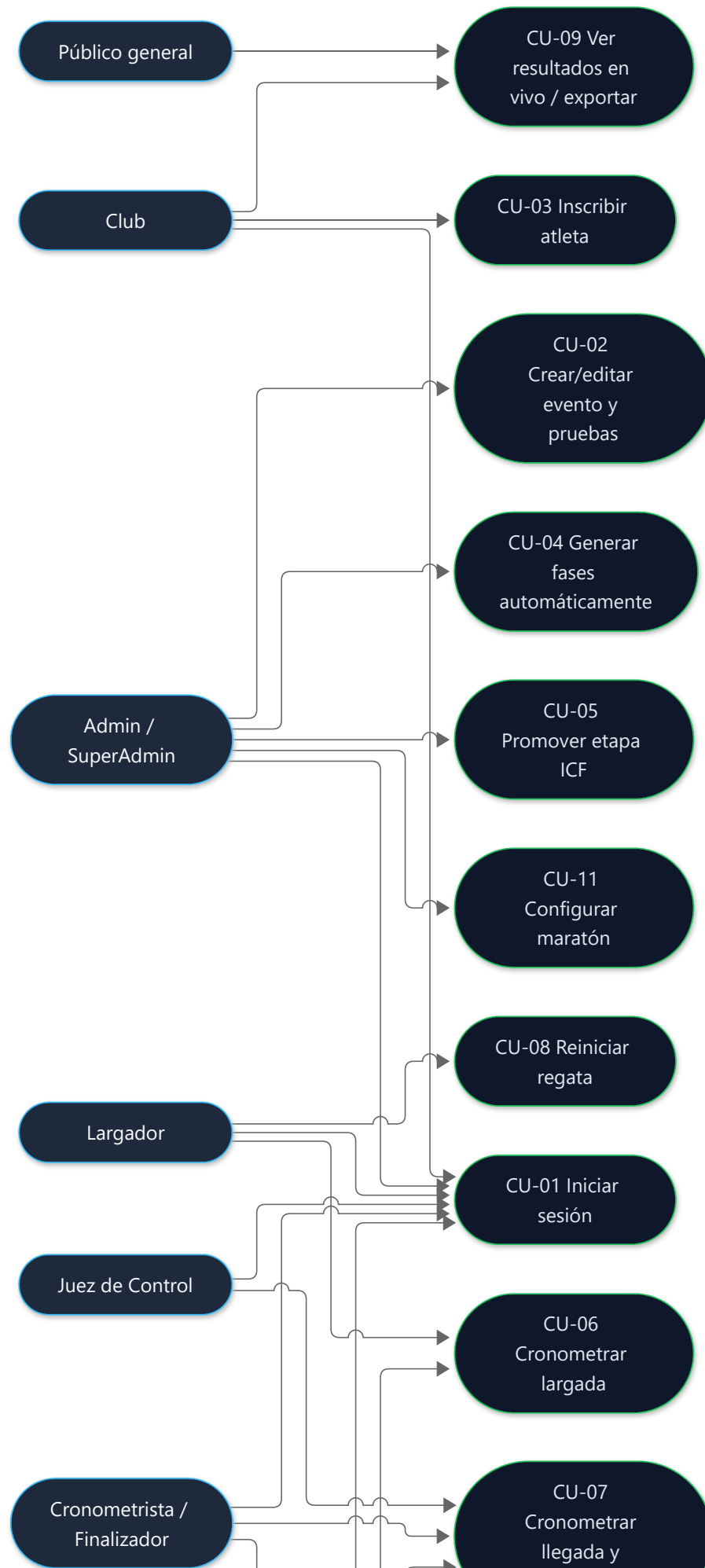
1. Introducción

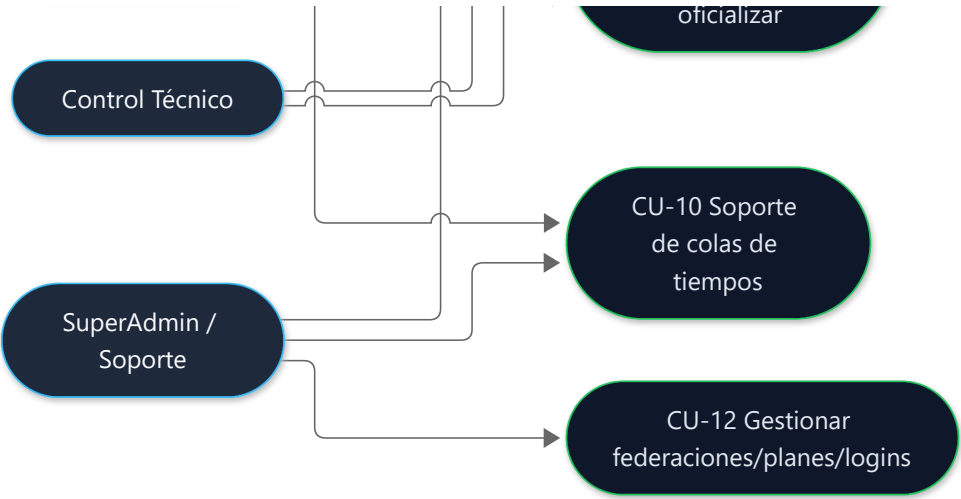
Este documento especifica los casos de uso del frontend SportTrack. Cada caso de uso se identifica con `CU-##` y se vincula a los requerimientos funcionales del [documento 01](#).

Convención de actores: se usa el rol tal como se muestra en la UI, aclarando en el encabezado el valor técnico cuando corresponde.

2. Diagrama de casos de uso

Mermaid no posee un tipo de diagrama UML de casos de uso nativo; se representa con un `flowchart`. Los actores se modelan como nodos a la izquierda/derecha y los casos de uso como nodos ovalados (`([...])`).





3. Catálogo de casos de uso

CU	Nombre	Actor principal	RF relacionados
CU-01	Iniciar sesión	Todos los autenticados	RF-57, RF-58, RF-60, RNF-12, RNF-13
CU-02	Crear / editar evento y configurar pruebas	Admin / SuperAdmin	RF-01...RF-06
CU-03	Inscribir atleta	Club / Admin	RF-07...RF-11
CU-04	Generar fases automáticamente	Admin / SuperAdmin	RF-12, RF-13, RF-14, RF-16, RF-18, RF-19
CU-05	Promover etapa ICF	Admin / SuperAdmin	RF-15
CU-06	Cronometrar largada	Largador / Control Técnico	RF-20, RF-25, RF-26
CU-07	Cronometrar llegada y validar/oficializar	Cronometrista / Juez de Control / Control Técnico	RF-21, RF-22, RF-23, RF-35, RF-36
CU-08	Reiniciar regata	Largador / Admin	RF-27
CU-09	Consultar resultados en vivo y exportar PDF/CSV	Público / Club / Admin	RF-33, RF-34, RF-54, RF-55
CU-10	Consola de soporte de colas de tiempos	Soporte / SuperAdmin	RF-30, RF-31, RF-56
CU-11	Configurar maratón	Admin / SuperAdmin	RF-37...RF-40
CU-12	Gestionar federaciones, planes y logins	SuperAdmin	RF-41...RF-45, RF-59

4. Especificación de casos de uso

CU-01 — Iniciar sesión

Campo	Valor
ID	CU-01
Actor(es)	Todos los actores autenticados
Objetivo	Autenticar al usuario, resolver su rol y plan, y aterrizarlo en su panel.
Precondiciones	Usuario registrado y activo en la API.
Postcondiciones	Sesión iniciada con token en <code>localStorage</code> ; plan restaurado; usuario redirigido a su panel por rol.
RF	RF-57, RF-58, RF-60 · RNF-12, RNF-13, RNF-15

Flujo principal

1. El usuario accede a `/login` e ingresa usuario y contraseña.
2. El sistema envía `POST /auth/login` con header `X-Client-App: sporttrack`.
3. La API valida credenciales y devuelve token, datos de usuario y plan; además setea la cookie `X-Access-Token`.
4. `AuthContext` normaliza el usuario y el plan (`normalizeAuthUser` , `normalizePlan`) y persiste token y datos en `localStorage`.
5. El sistema resuelve la ruta destino con `getDashboardPathForRole(rol)`.
6. El sistema redirige al panel correspondiente (`/super` , `/club` , `/juez-control` , `/jueces/largador` , `/jueces/llegada` , `/control-tecnico`).
7. Al reanudar, `AuthContext` valida la sesión contra `GET /auth/me` y restaura el plan.

Flujos alternativos / excepciones

- **A1 — Credenciales inválidas:** la API responde `401` ; el sistema muestra el error y no inicia sesión.
- **A2 — Usuario inactivo:** la API rechaza; se muestra el mensaje correspondiente.
- **A3 — Sesión expirada:** ante un `401` en cualquier request, el interceptor limpia `USER_DATA` y `AUTH_TOKEN` ; el usuario es redirigido a `/login`.
- **A4 — Rol sin plan compatible:** `ProtectedRoute` delega en `PlanGuard` , que muestra la pantalla de bloqueo.
- **A5 — Alias soporte_tecnico:** `isSuperAdminUser` lo trata como `SuperAdmin` y omite restricciones de plan.

CU-02 — Crear / editar evento y configurar pruebas

Campo	Valor
ID	CU-02
Actor(es)	Admin / SuperAdmin
Objetivo	Dar de alta o modificar un evento y su grilla de pruebas/horarios.
Precondiciones	Sesión con rol Admin/SuperAdmin y acceso al sistema SportTrack.
Postcondiciones	Evento persistido; pruebas y horarios configurados; cronograma actualizado.
RF	RF-01...RF-06

Flujo principal

1. El Admin entra a `Eventos` y presiona "Nuevo evento".
2. Completa `EventForm` (nombre, fechas, ubicación, federación, estado de inscripciones) y guarda.
3. El sistema persiste vía `POST /eventos` y muestra el evento en la grilla (`GestionEventosSection`).
4. El Admin abre `ConfigurarPruebasModal` y agrega pruebas (categoría, bote, distancia, sexo), incluyendo múltiples categorías.
5. El sistema persiste las pruebas y genera el cronograma con horarios (`SchedulerService`).
6. El Admin ajusta horarios; `useResultados.handleUpdateFaseHorario` propaga el cambio con `FaseService.batchUpdate`.

Flujos alternativos / excepciones

- **A1 — Edición de fecha:** al cambiar la fecha del evento, el cronograma se recalcula sobre la nueva fecha.
- **A2 — Evento "todo manual":** al activarlo, se deshabilitan consolas de juez y se habilita la carga manual (`isManualTiming`).
- **A3 — Sin permiso de imágenes:** si el plan no habilita `permitirCargaImagenes` , la carga de imágenes queda deshabilitada.
- **A4 — Datos inválidos:** la validación del formulario impide guardar.

CU-03 — Inscribir atleta

Campo	Valor
ID	CU-03
Actor(es)	Club (principal), Admin
Objetivo	Inscribir atletas/botes en una prueba de un evento.
Precondiciones	Evento con inscripciones abiertas; atleta perteneciente al club (o disponible).
Postcondiciones	Inscripción registrada; disponible para siembra y fases.

Campo	Valor
RF	RF-07...RF-11

Flujo principal

1. El Club entra a *Inscripciones* y busca el evento/prueba.
2. Abre `InscripcionAtletaModal` y selecciona el atleta o arma el bote de equipo con tripulantes.
3. `inscripcionEligibilityUtils` valida la elegibilidad.
4. El sistema envía `POST /inscripciones` (o `/inscripciones/registro`).
5. La inscripción aparece en la lista y queda disponible para siembra.
6. Si corresponde, se registra el pago (`RegistrarPagoModal` → `PagoService`).

Flujos alternativos / excepciones

- **A1 — Elegibilidad fallida:** el sistema rechaza la inscripción y explica el motivo.
- **A2 — Cupo/maxAtletas alcanzado:** el plan impone el límite (si no es `-1`).
- **A3 — Incripciones cerradas:** la UI deshabilita la acción.

CU-04 — Generar fases automáticamente

Campo	Valor
ID	CU-04
Actor(es)	Admin / SuperAdmin
Objetivo	Generar las etapas y series de un EventoPrueba con siembra de carriles.
Precondiciones	EventoPrueba con inscripciones confirmadas.
Postcondiciones	Fases creadas con series, carriles y progresión ICF lista.
RF	RF-12, RF-13, RF-14, RF-16, RF-17, RF-18, RF-19

Flujo principal

1. El Admin entra a la gestión de resultados del EventoPrueba.
2. El sistema muestra las inscripciones sin sembrar.
3. El Admin ejecuta "Generar fases" (`FaseService.generar` → `POST /fases/Generar/{id}`).
4. El motor de progresión (`ProgressionEngine` , `promotionHelpers`) reparte atletas en series y carriles.
5. Las `FaseCard` muestran las etapas generadas; el cronograma se actualiza.
6. El Admin puede editar horarios/carriles (`batchUpdate`) o eliminar una fase (`DELETE /fases/{id}`).

Flujos alternativos / excepciones

- **A1 — Generación manual:** `useResultados.handleGenerarManual` → `POST /fases/GenerarManual/{id}` con colocaciones explícitas.
- **A2 — Siembra insuficiente:** si hay menos inscripciones que carriles requeridos, el sistema ajusta las series.
- **A3 — Auditoría de progresión:** `GET /fases/ProgresionAudit/{id}` expone el detalle para revisión.

CU-05 — Promover etapa ICF

Campo	Valor
ID	CU-05
Actor(es)	Admin / SuperAdmin
Objetivo	Promover los clasificados de una etapa a la siguiente según el reglamento ICF.
Precondiciones	Etapa con resultados cargados y validados.
Postcondiciones	Atletas clasificados asignados a la etapa siguiente; progresión auditable.
RF	RF-15

Flujo principal

1. El Admin abre la etapa finalizada (p. ej. Eliminatorias).
2. Ejecuta "Promover" (`FaseService.promover` → `POST /fases/Promover/{id}`).

3. El backend aplica la regla ICF de clasificación (primeros N por serie, mejores tiempos, repescas si aplica).
4. Los clasificados se asignan a la etapa siguiente; los demás quedan fuera.
5. El sistema registra la acción en la auditoría de progresión (`ProgressionAuditPage`).

Flujos alternativos / excepciones

- **A1 — Resultados incompletos:** si faltan tiempos, el sistema impide promover.
- **A2 — Empate en la última posición clasificatoria:** se aplica el criterio de desempate de la regla ICF.

CU-06 — Cronometrar largada

Campo	Valor
ID	CU-06
Actor(es)	Largador (principal), Control Técnico
Objetivo	Registrar el t0 oficial de una regata y notificar a los conectados.
Precondiciones	Sesión con <code>accesoControlesLive</code> ; serie en estado listo.
Postcondiciones	t0 registrado (o encolado); cronometrista notificado.
RF	RF-20, RF-25, RF-26 · RNF-11, RNF-14

Flujo principal

1. El Largador entra a `/jueces/largador` y selecciona el evento y la serie.
2. El sistema sincroniza el reloj (5 muestras, `GetServerTime`) y calcula `serverOffset`.
3. El Largador presiona "Largar"; se toma el instante del click como t0 (`getSyncedNow`).
4. `TimingSignalRService.deliverRaceStart` intenta: **SignalR** (`RequestStartRace`).
5. Si SignalR falla, cae a **HTTP** (`FaseService.iniciar`).
6. Si ambos fallan, se **encola** el t0 (`savePendingRaceStart`) y se reintenta cada 4 s.
7. El cronometrista recibe `RaceStarted` y se reproduce el timbre (`playRaceStartBell`).

Flujos alternativos / excepciones

- **A1 — Sin conexión:** se aplica el paso 6; el t0 nunca se regenera.
- **A2 — Evento global:** `globalRaceStarted` notifica a los suscriptores globales.
- **A3 — Largada "catch-up":** `isFreshRaceStart` evita reproducir el timbre al abrir una carrera antigua.

CU-07 — Cronometrar llegada y validar/oficializar

Campo	Valor
ID	CU-07
Actor(es)	Cronometrista/Finalizador, Juez de Control, Control Técnico
Objetivo	Capturar tiempos de llegada, revisarlos y oficializarlos.
Precondiciones	Regata largada; resultados/inscripciones disponibles.
Postcondiciones	Resultados oficiales publicados en la pizarra pública.
RF	RF-21, RF-22, RF-23, RF-35, RF-36 · RNF-09, RNF-10

Flujo principal

1. El Cronometrista entra a `/jueces/llegada` y selecciona evento y serie.
2. Al recibir `RaceStarted`, el sistema habilita la captura y arranca el cronómetro.
3. Por cada carril, el Cronometrista registra el tiempo (`sendTime` / cola local).
4. Los tiempos se envían (`SendTime`) y se reflejan en la pizarra (`TimeReceived`).
5. La fase se marca "en revisión" (`RaceInReview`) mediante `enviarARevision`.
6. El Juez de Control revisa y oficializa (`updateResultStatus`); se emite `globalRaceOfficialized`.
7. La pizarra pública muestra los resultados oficiales.

Flujos alternativos / excepciones

- **A1 — Conexión caída:** los tiempos van a la cola local; puede descargarse el **PDF de respaldo**.
- **A2 — Tiempo pendiente en outbox:** Soporte lo confirma desde CU-10.

- **A3 — Corrección de tiempo:** el Juez de Control puede reabrir/editar antes de oficializar.
- **A4 — Carga manual:** en eventos "todo manual" o planes sin controles live se usa `ManualTiming` o la carga manual de resultados.

CU-08 — Reiniciar regata

Campo	Valor
ID	CU-08
Actor(es)	Largador, Admin
Objetivo	Anular y repetir una regata por incidente operativo.
Precondiciones	Regata en curso o largada en falso.
Postcondiciones	Regata reiniciada; conectados notificados; motivo auditado.
RF	RF-27 · RN-10

Flujo principal

1. El Largador abre `ReiniciarFaseDialog` sobre la regata.
2. Selecciona una categoría de `REINICIAR_FASE_CATEGORIAS` (mala largada, postergación, problema técnico, problema externo, otro).
3. Si elige "Otro", ingresa un detalle de al menos 10 caracteres (`isReiniciarMotivoValid`).
4. Confirma el reinicio; el sistema llama `requestResetRace` (`RequestResetRace`).
5. Se emite el evento `RaceReset` a los conectados.
6. La serie vuelve a estado listo para una nueva largada.
7. La acción queda registrada en la auditoría.

Flujos alternativos / excepciones

- **A1 — Sin conexión:** el sistema informa que no hay conexión activa con el servidor de tiempos.
- **A2 — Motivo inválido:** "Otro" sin detalle suficiente bloquea la confirmación.
- **A3 — Uso indebido:** la UI recuerda con `REINICIAR_FASE_AVISO` que no debe usarse para corregir siembras ni cronograma.

CU-09 — Consultar resultados en vivo y exportar PDF/CSV

Campo	Valor
ID	CU-09
Actor(es)	Público general, Club, Admin
Objetivo	Visualizar resultados en tiempo real y exportarlos.
Precondiciones	Evento existente. El acceso público no requiere login.
Postcondiciones	Resultados visualizados; archivo descargado si se solicita.
RF	RF-33, RF-34, RF-35, RF-54, RF-55

Flujo principal

1. El actor abre `/resultados/:id` (evento) o es dirigido desde un enlace compartido.
2. El sistema carga el evento y sus fases (`EventoService` , `FaseService`).
3. `TimingSignalRService` se conecta al grupo del evento y escucha `globalTimeReceived` , `globalRaceStarted` , `globalRaceInReview` , `globalRaceOfficialized` .
4. `resultadosHelpers.computePositionsForPhase` calcula posiciones y diferencias.
5. La tabla (`ResultadosTable`) se actualiza en tiempo real.
6. El actor exporta: `PdfExportService` (cronograma, programa, fase, start list, respaldo) o `CsvExportService` .

Flujos alternativos / excepciones

- **A1 — Evento maratón:** los resultados se agrupan por clasificación.
- **A2 — Sin conexión SignalR:** el sistema muestra el último estado cargado por REST y reintentará conectar.
- **A3 — Exportación no habilitada:** si el plan no incluye `exportacionPdf` , el botón no se ofrece.

CU-10 — Consola de soporte de colas de tiempos

Campo	Valor
ID	CU-10
Actor(es)	SuperAdmin / Soporte técnico
Objetivo	Inspeccionar y resolver las colas temporales (outbox) de tiempos pendientes.
Precondiciones	Sesión SuperAdmin; existen entradas pendientes en el outbox del servidor.
Postcondiciones	Cola confirmada (consolidada) o descartada.
RF	RF-30, RF-31, RF-56

Flujo principal

1. Soporte entra a `SoporteSection` y abre la pestaña "Colas temporales".
2. El sistema consulta `SupportService.getTimingOutbox()` → `GET /support/timing-outbox`.
3. Se listan las entradas con su fase y usuario.
4. Soporte selecciona una entrada y presiona "Confirmar" → `POST /support/timing-outbox/{faseId}/commit`.
5. El sistema consolida los tiempos en la fase y recarga la cola con un toast de éxito.

Flujos alternativos / excepciones

- **A1 — Descartar:** `DELETE /support/timing-outbox/{id}` previa confirmación (`ConfirmDialog`).
- **A2 — Error de carga:** se muestra toast de error y se permite reintentar.

CU-11 — Configurar maratón

Campo	Valor
ID	CU-11
Actor(es)	Admin / SuperAdmin
Objetivo	Configurar un evento en modalidad maratón y generar su largada masiva.
Precondiciones	Evento creado con inscripciones agrupables.
Postcondiciones	Fase "Largada" de maratón creada; programa/start list disponible.
RF	RF-37...RF-40 · RN-13

Flujo principal

1. El Admin abre `ConfigurarMaratonModal`.
2. Define los grupos y la configuración de largada.
3. Selecciona los EventoPrueba que participan.
4. El sistema llama `FaseService.generarLargadaMaraton` → `POST /fases/GenerarLargadaMaraton`.
5. Se crea una fase "Largada" con todos los inscriptos del grupo.
6. `maratonStartListUtils` y `MaratonProgramaList` arman el programa/start list.
7. El Admin exporta el programa con `exportProgramaMaraton` / `exportMaratonLargadaStartList`.

Flujos alternativos / excepciones

- **A1 — Sin inscriptos:** el sistema impide generar la largada.
- **A2 — Resultados:** `groupMaratonResultadosByClasificacion` agrupa los resultados por clasificación.
- **A3 — Cronograma:** `maratonScheduleUtils` calcula los horarios de la modalidad.

CU-12 — Gestionar federaciones, planes y logins

Campo	Valor
ID	CU-12
Actor(es)	SuperAdmin
Objetivo	Administrar el SaaS: federaciones, clubes, planes y usuarios.
Precondiciones	Sesión SuperAdmin (o alias <code>soporte_tecnico</code>).
Postcondiciones	Federaciones/clubes/planes/logins actualizados; accesos coherentes con el plan.

Campo	Valor
RF	RF-41...RF-45, RF-59

Flujo principal

1. El SuperAdmin entra a `GestionFederacionesSection` y administra federaciones (`FederacionService`).
2. En `SaaSManagement` asigna planes y consulta estado/métricas (`SaaSService`).
3. Activa/desactiva clubes (`/saas/clubes/{id}/toggle-activo`).
4. En `GestionLoginsSection` crea usuarios por rol; el sistema valida el plan:
 - login de club → requiere `accesoDashboardClub` ;
 - login de juez → requiere `accesoControlesLive` .
5. El sistema persiste el rol (`rolFederacion`) y sus permisos por evento.

Flujos alternativos / excepciones

- **A1 — Plan insuficiente:** no se ofrece crear el login correspondiente.
- **A2 — Club sin federación:** el registro de un login de club requiere club válido (RN-07).
- **A3 — Backup:** desde `BackupsSection` se ejecutan respaldos (`BackupService`).

Pie del pack

Documento anterior	Documento actual	Documento siguiente
← 01 Requerimientos	02 — Casos de Uso	03 — Diagramas de Flujo →

[README](#) · [01](#) · [02](#) · [03](#) · [04](#) · [05](#) · [06](#)

03 — Diagramas de Flujo

Proyecto: SportTrack — Frontend de Competencias **Última actualización:** 2026-09-27 **Notación:** Mermaid (`flowchart` , `stateDiagram-v2` , `sequenceDiagram`)

Índice

- 1. Introducción
- 2. (a) Alta/edición de evento y configuración de pruebas
- 3. (b) Siembra/sorteo y generación de fases
- 4. (c) Progresión ICF
- 5. (d) Ciclo de vida de una regata
- 6. (e) Cronometraje con resiliencia offline
- 7. (f) Autenticación/autorización por rol y plan
- 8. (g) Flujo maratón
- 9. (h) Auditoría de incidencias/errores
- 10. Pie del pack

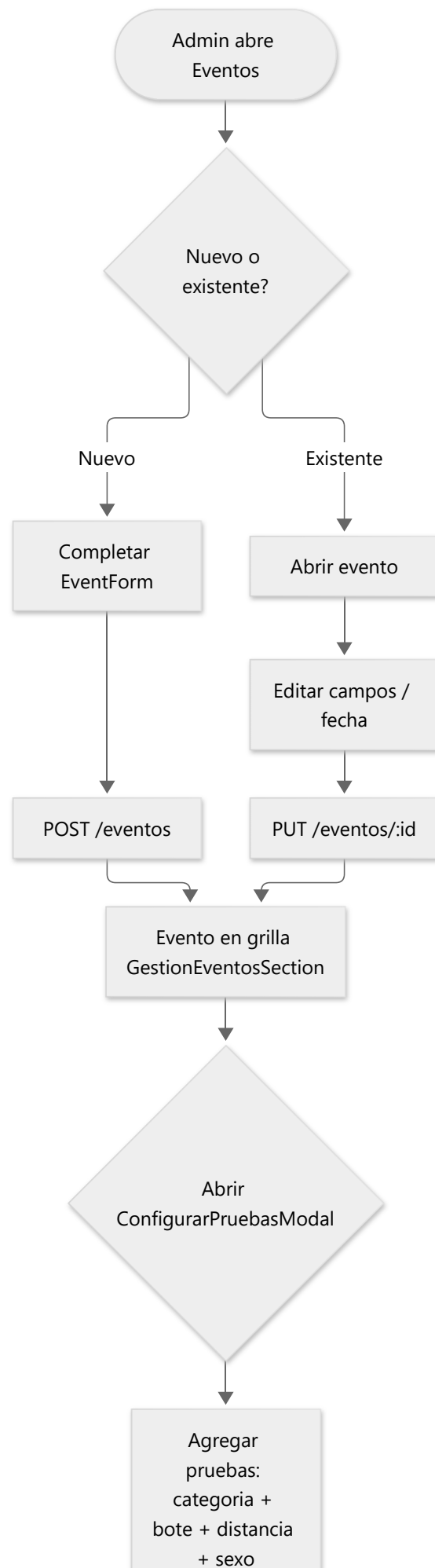
1. Introducción

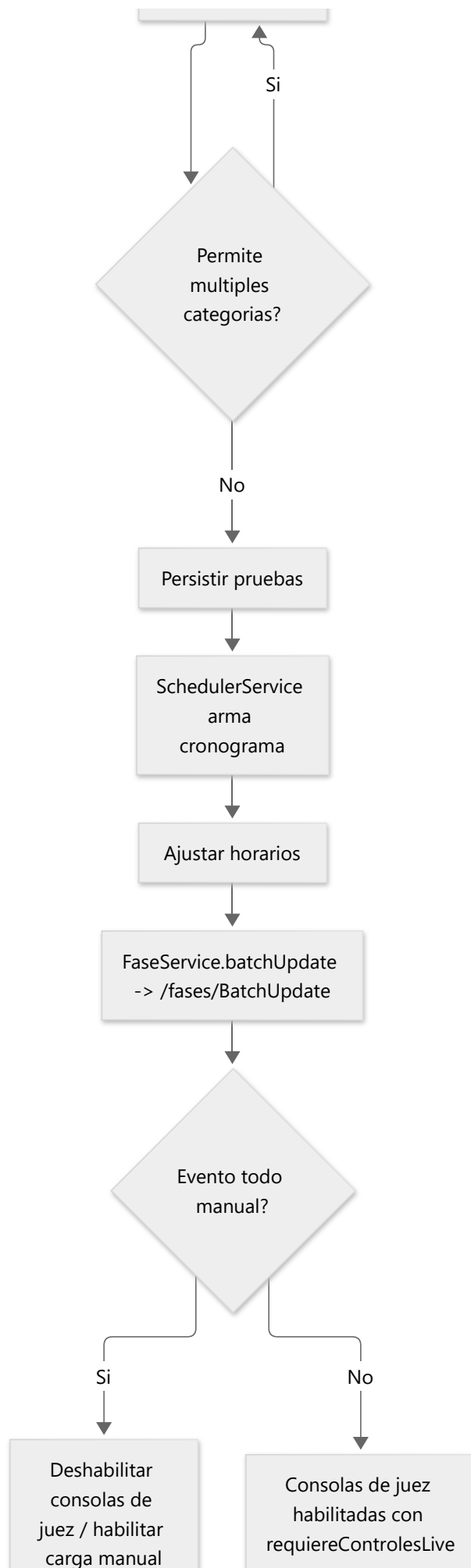
Este documento presenta los flujos operativos y técnicos principales del frontend SportTrack mediante diagramas Mermaid reproducibles. Cada diagrama se vincula con los requerimientos ([doc 01](#)) y los casos de uso ([doc 02](#)).

Los diagramas son **válidos** para el renderizador Mermaid de GitHub/VSCode. Para copiarlos, usá el bloque completo incluido el encabezado del tipo de diagrama.

(a) Alta/edición de evento y configuración de pruebas

RF: RF-01...RF-06 · **CU:** CU-02



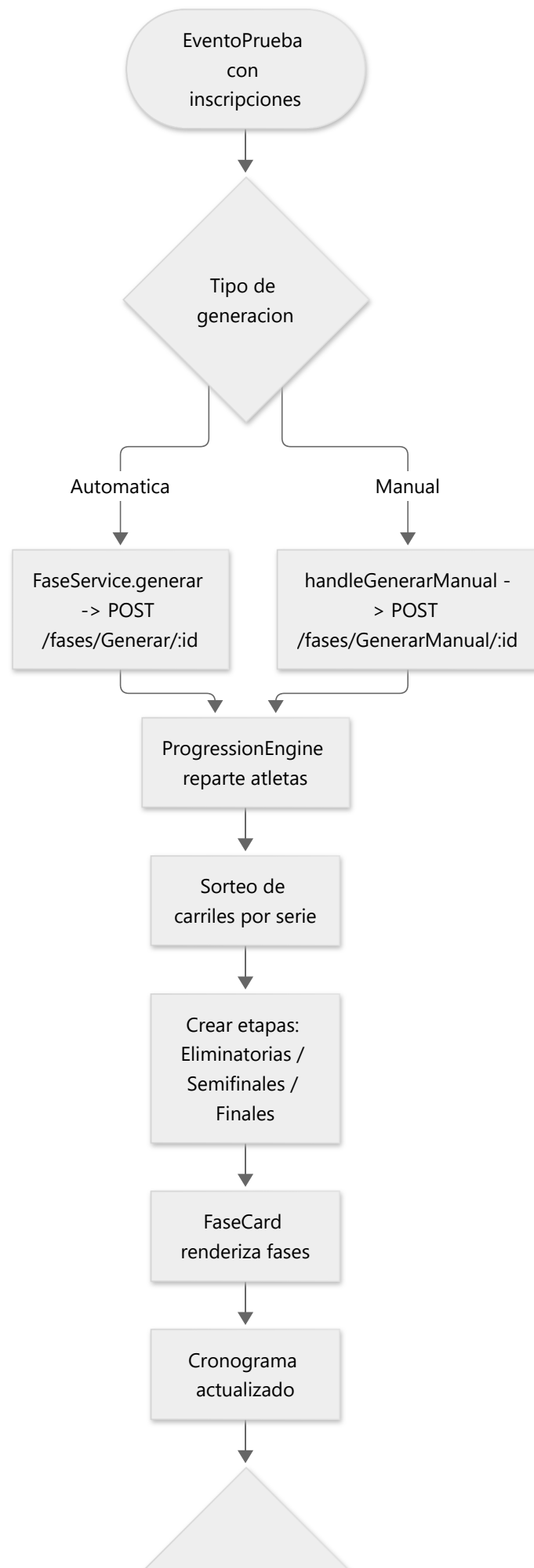


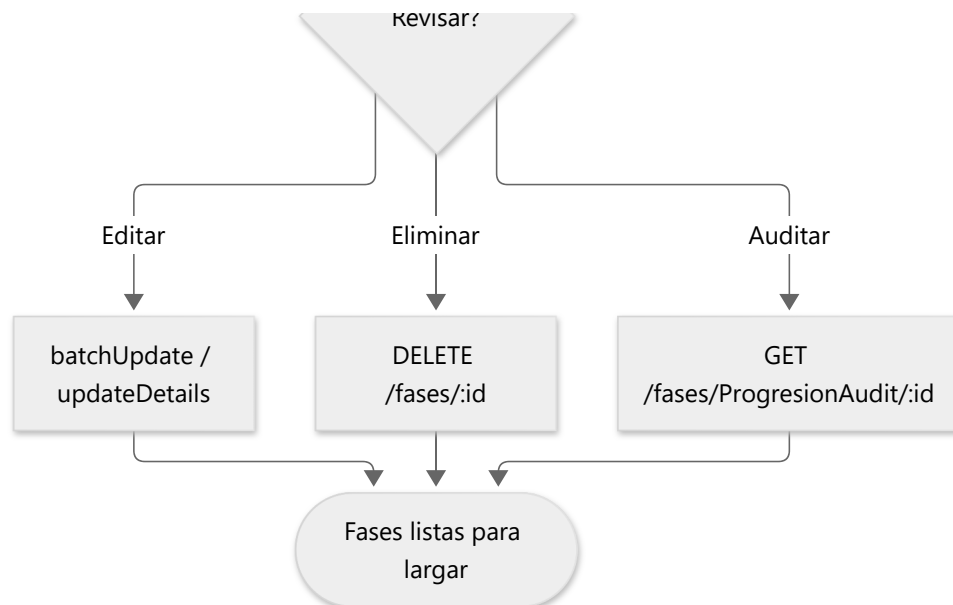


Notas. La edición de la fecha del evento recalcula el cronograma. La carga de imágenes depende del flag `permitirCargaImagenes` del plan.

(b) Siembra/sorteo y generación de fases

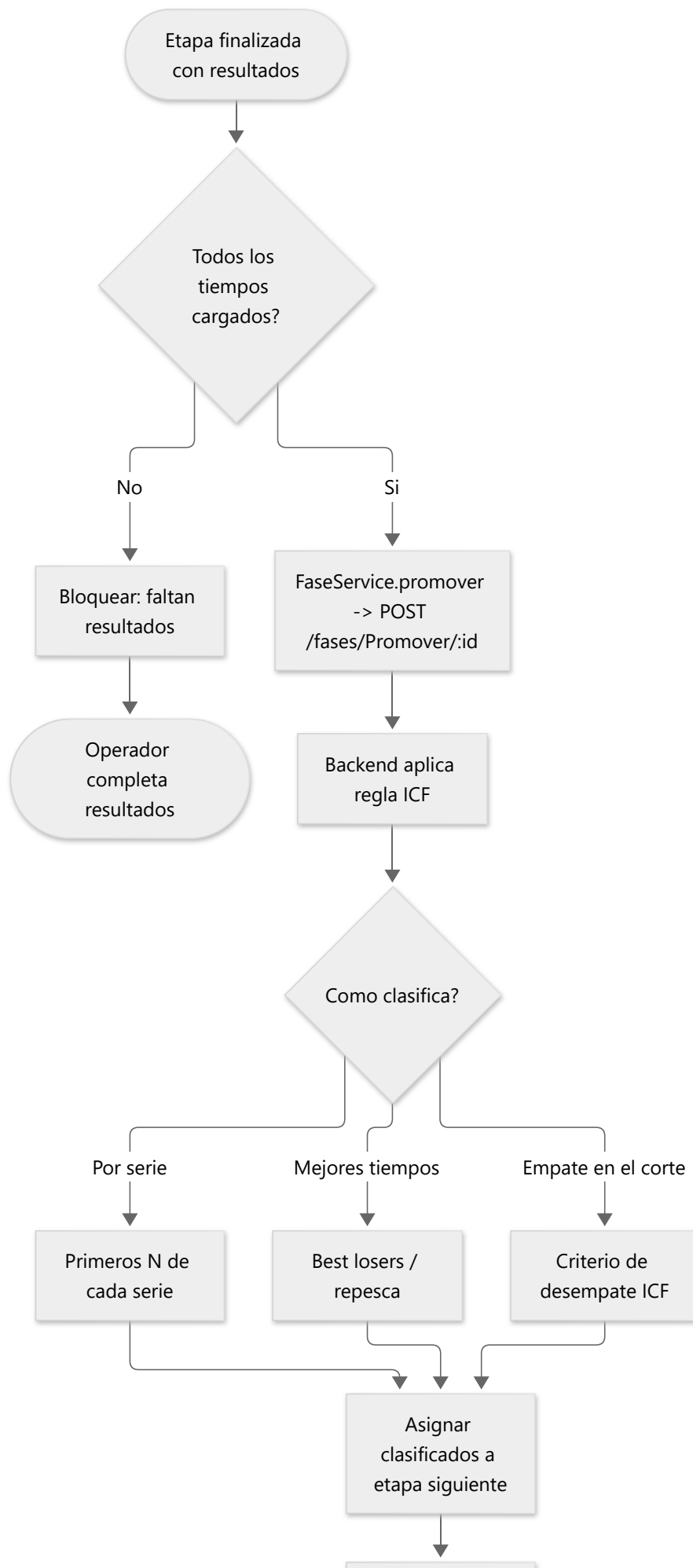
RF: RF-12, RF-13, RF-14 · **CU:** CU-04





(c) Progresión ICF

RF: RF-15, RF-19 · **CU:** CU-05 · Reglamento ICF de canotaje sprint.

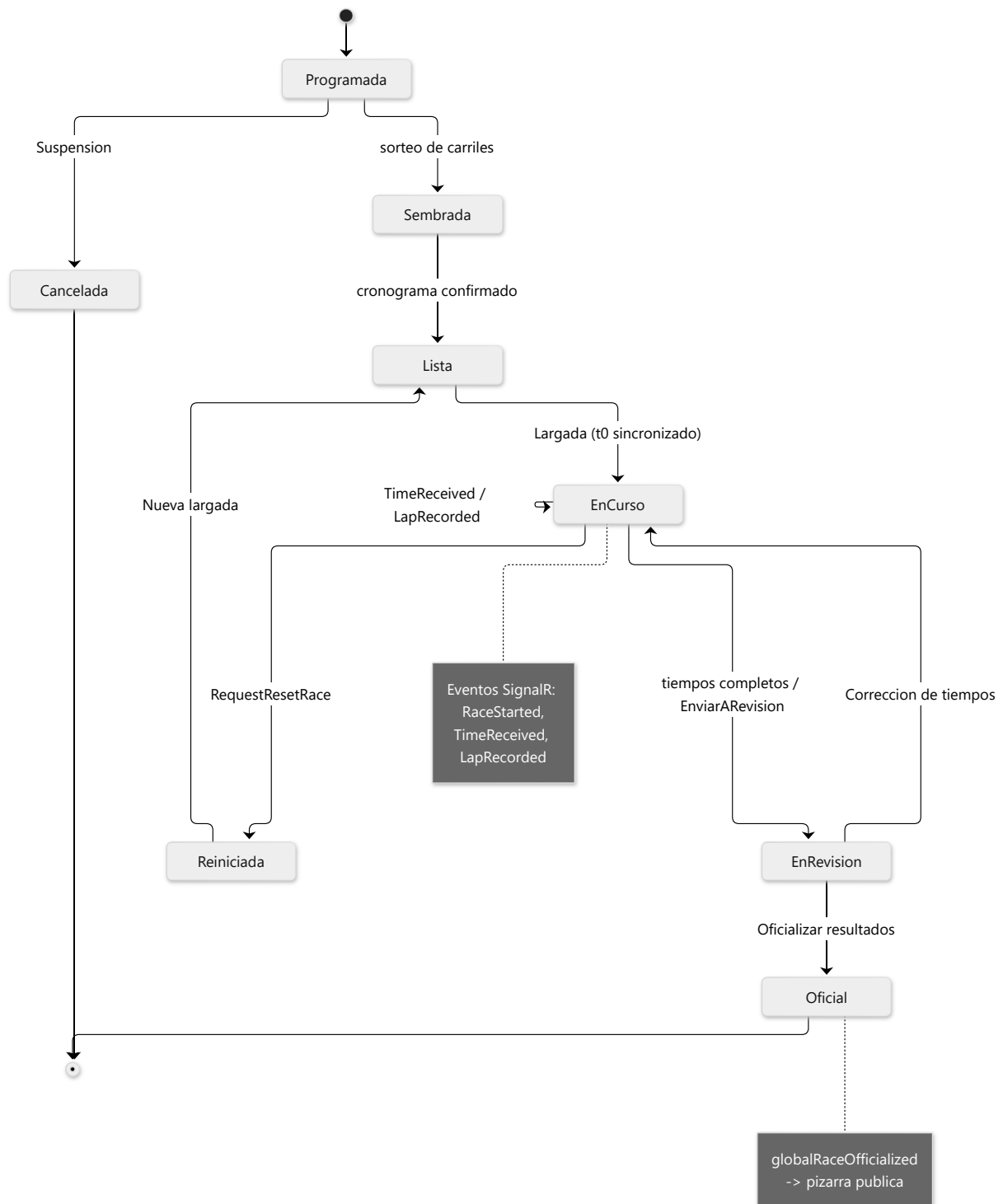




Notas. La progresión se ejecuta en el backend; el frontend solo la dispara y muestra el resultado y su auditoría.

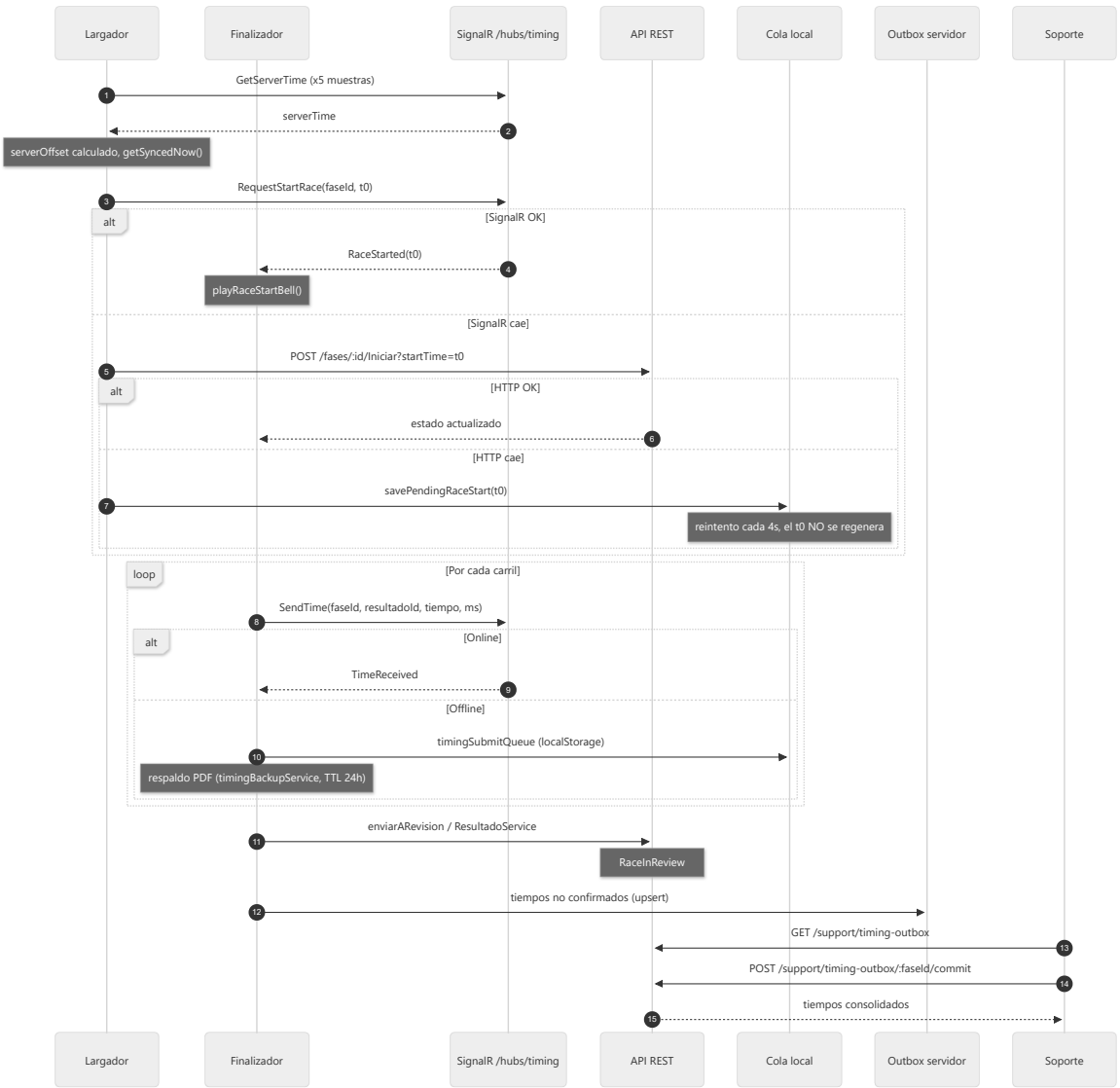
(d) Ciclo de vida de una regata

RF: RF-20, RF-21, RF-22, RF-27 · **CU:** CU-06, CU-07, CU-08



(e) Cronometraje con resiliencia offline

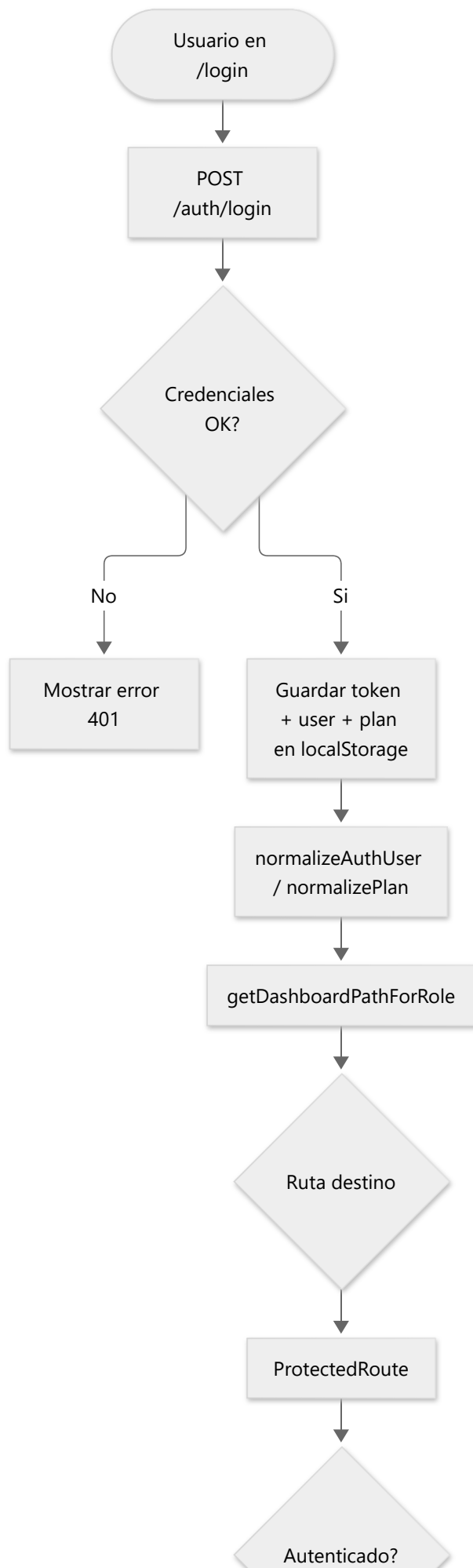
RF: RF-25, RF-28, RF-29, RF-30, RF-31, RF-32 · **CU:** CU-06, CU-07, CU-10

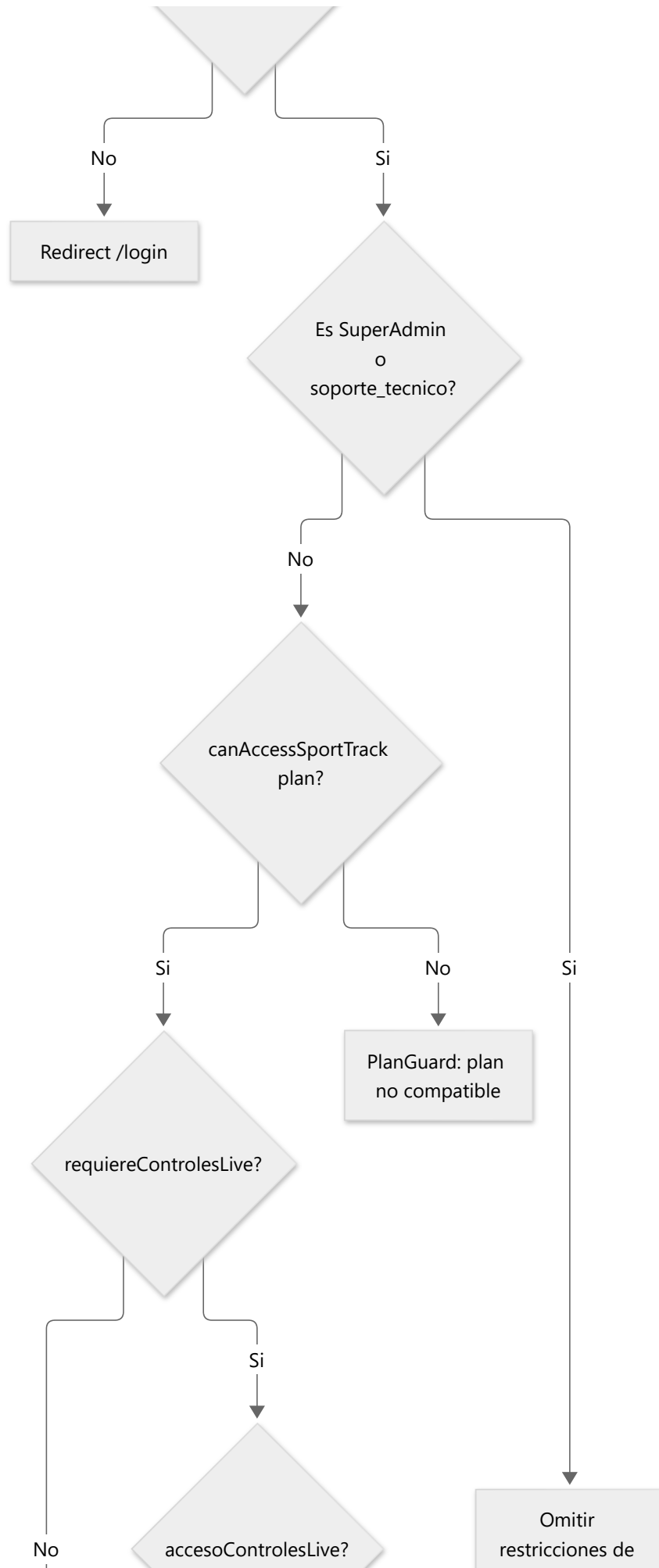


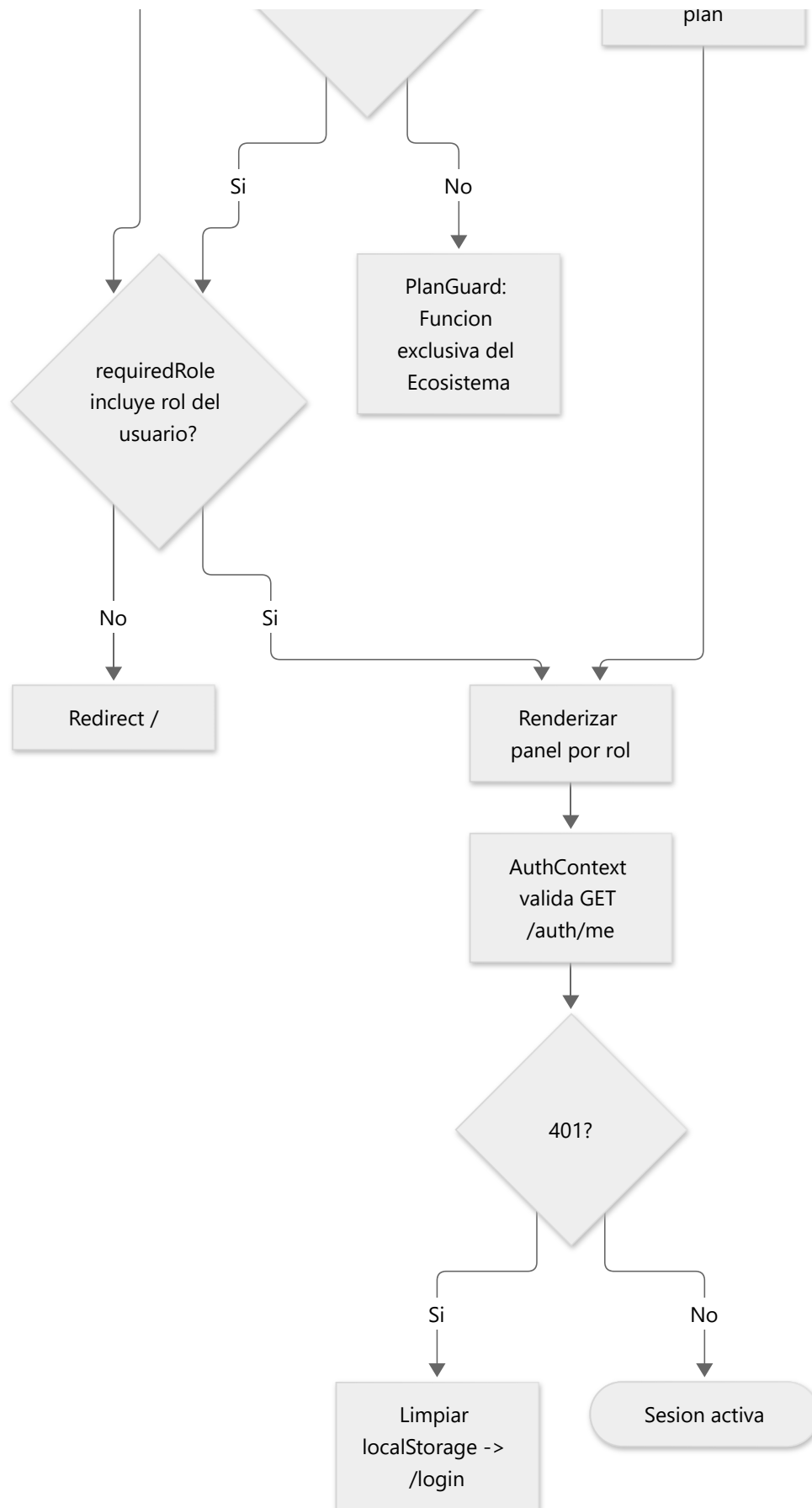
Notas. El outbox del servidor (/timing-outbox/*) y la consola de Soporte permiten recuperar capturas que no llegaron a tiempo. La cola local (sporttrack_pending_timing_saves) y el respaldo PDF cubren el caso extremo de corte prolongado.

(f) Autenticación/autorización por rol y plan

RF: RF-45, RF-57, RF-58 · **RNF:** RNF-12, RNF-13 · **CU:** CU-01, CU-12

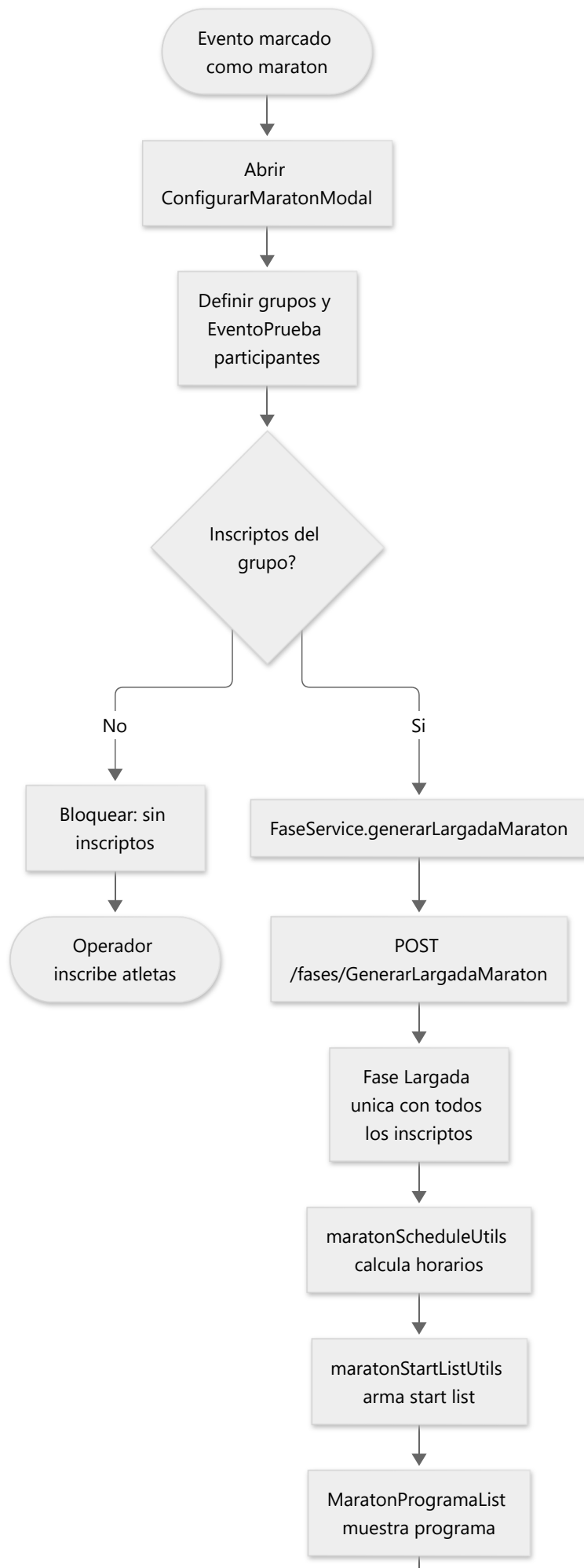


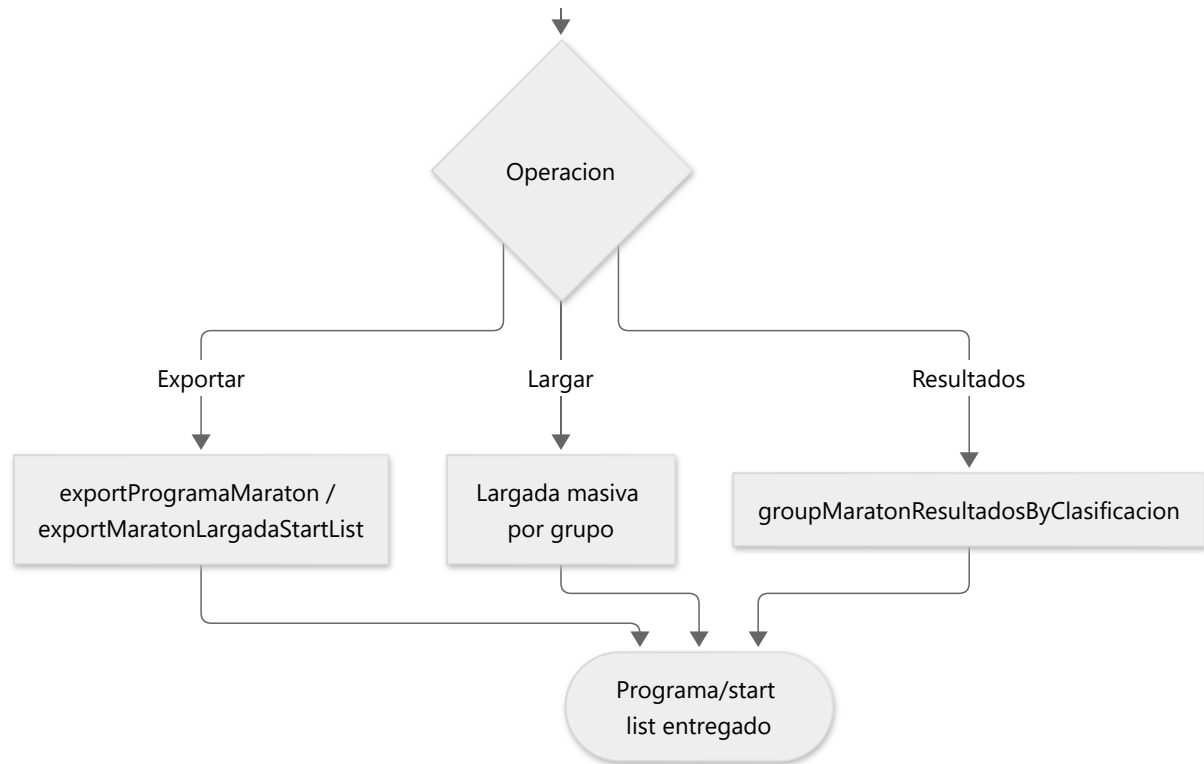




(g) Flujo maratón

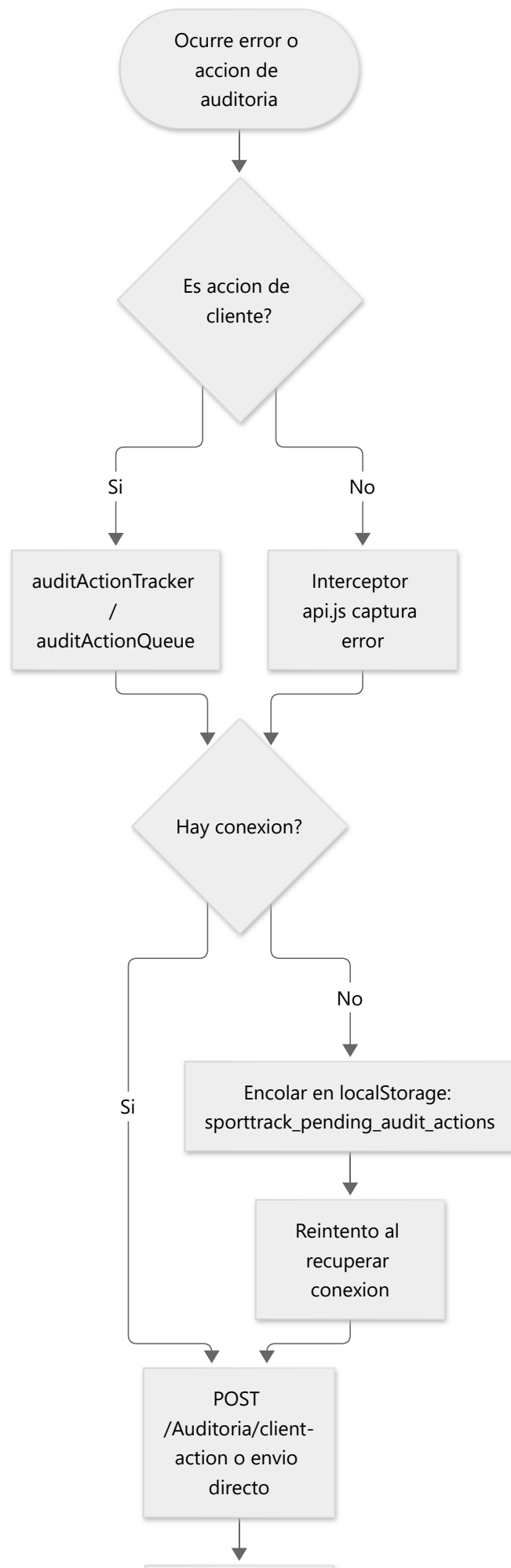
RF: RF-37...RF-40 · CU: CU-11

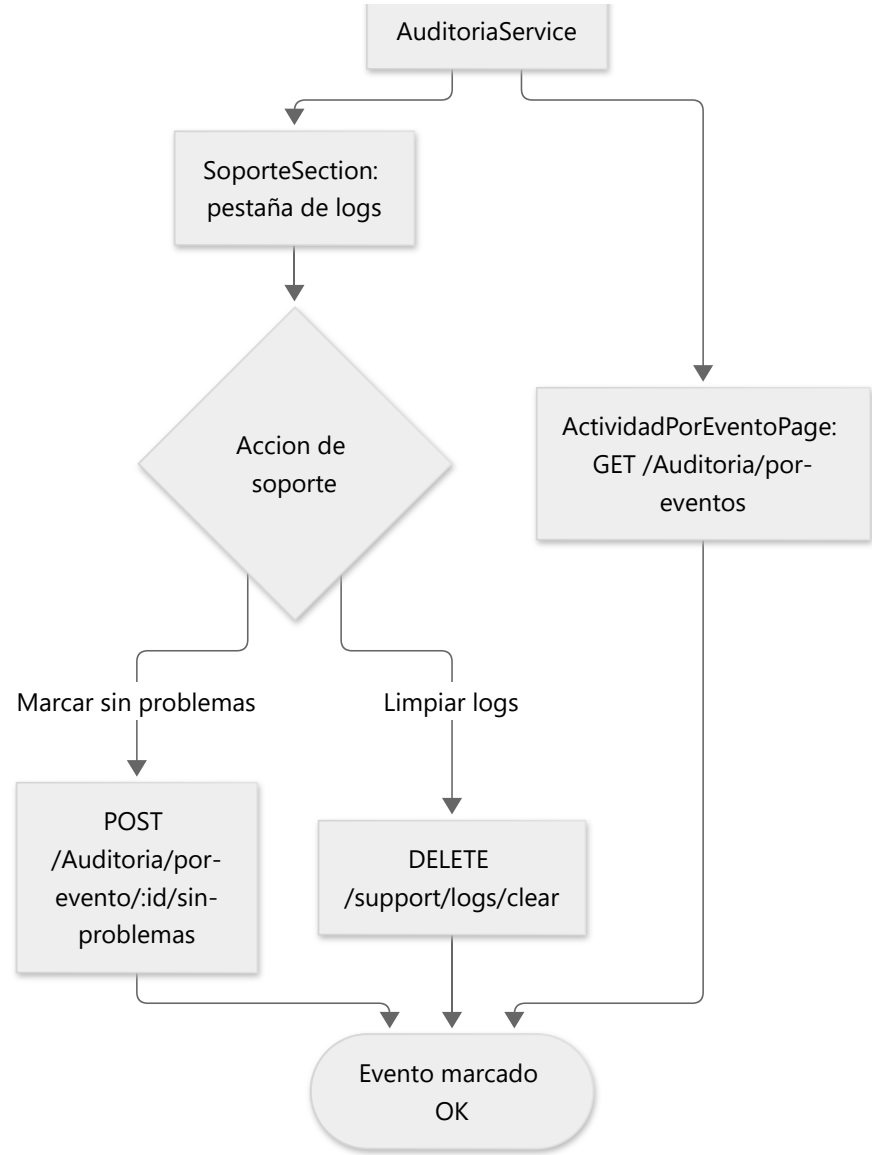




(h) Auditoría de incidencias/errores

RF: RF-46...RF-49 · **CU:** CU-12 · **RNF:** RNF-15, RNF-21





Pie del pack

Documento anterior	Documento actual	Documento siguiente
← 02 Casos de Uso	03 — Diagramas de Flujo	04 — Manual de Usuario →

04 — Manual de Usuario

Proyecto: SportTrack — Frontend de Competencias **Última actualización:** 2026-09-27 **Audiencia:** usuarios finales y operadores de competencia

Índice

- 1. Introducción
- 2. Acceso y navegación
- 3. Público general
- 4. Club
- 5. Administrador de Federación / Admin
- 6. Largador
- 7. Cronometrista / Finalizador
- 8. Juez de Control
- 9. Control Técnico
- 10. SuperAdmin / Soporte
- 11. FAQ y resolución de problemas
- 12. Pie del pack

1. Introducción

Este manual explica, por rol, cómo operar SportTrack. Cada sección incluye pasos numerados, tablas de campos, notas y advertencias. Si una función no aparece en tu menú, probablemente tu **rol** o **plan SaaS** no la habilita (ver sección 5.6 y FAQ).

Roles cubiertos: Público, Club, Admin/Federación, Largador, Cronometrista (rotulado "Finalizador"), Juez de Control, Control Técnico, SuperAdmin/Soporte.

2. Acceso y navegación

2.1 Ingresar al sistema

- 1. Abrió el navegador en la URL de la aplicación (o la app Android en tu tablet).
- 2. En `/login` ingresá **usuario** y **contraseña**.
- 3. Presioná **Iniciar sesión**. El sistema te lleva automáticamente a tu panel según tu rol:

Rol	Panel al ingresar
Admin / SuperAdmin	<code>/super</code>
Club	<code>/club</code>
Control Técnico	<code>/control-tecnico</code>
Largador	<code>/jueces/largador</code>
Cronometrista (Finalizador)	<code>/jueces/llegada</code>
Juez de Control	<code>/juez-control</code>

2.2 Elementos comunes de la interfaz

Elemento	Función
Barra lateral	Navegación entre módulos del panel.
Campana de notificaciones	Requiere acción (competencia/pagos), Mensajes, Novedades.
Toasts	Confirmaciones y errores breves en pantalla.
Tema claro/oscuro	Se guarda en el navegador y persiste entre sesiones.
Cerrar sesión	Termina la sesión y limpia los datos locales.

Nota. Si durante una operación se pierde la conexión, la app **no pierde tus datos**: se guardan localmente y se sincronizan al volver la conexión (ver sección 7).

3. Público general

No requiere usuario ni contraseña.

1. Abrí el enlace público de resultados: `/resultados/:id` (donde `:id` es el evento).
2. Vas a ver las **series y resultados en vivo**, agrupados por prueba/etapa.
3. La tabla se actualiza automáticamente cuando hay nuevos tiempos u oficializaciones.
4. Podés cambiar de serie/prueba con los selectores de la parte superior.
5. Si el organizador lo habilitó, podés descargar el **PDF del cronograma o del programa**.

Campo visible	Significado
**Carril (L1...)	Número de carril del bote
Posición (1º, 2º...)	Posición calculada en la serie
Tiempo	Tiempo oficial o provisional
Diferencia (+Xs)	Diferencia respecto del primero de la serie
Estado	Provisional / En revisión / Oficial

Nota. En eventos de **maratón** los resultados se muestran agrupados por clasificación.

4. Club

4.1 Inscribir un atleta

1. Entrá a **Inscripciones**.
2. Elegí el **evento** y la **prueba**.
3. Presioná **Inscribir atleta**.
4. En la ventana, elegí el atleta o armá el bote de equipo (agregando tripulantes si es K2, K4, C2, etc.).
5. El sistema **valida la elegibilidad** y muestra cualquier observación.
6. Confirmá: la inscripción queda en la lista.

Campo	Descripción	Obligatorio
Evento	Competencia destino	Sí
Prueba	Categoría + bote + distancia + sexo	Sí
Atleta / Tripulantes	Participantes del bote	Sí
Club	Club al que representa	Sí (precargado)

4.2 Ver resultados de tus atletas

1. Entrá a **Resultados** o abrí `/resultados/:id`.
2. Filtrá por prueba o serie.

4.3 Pagos

1. Entrá a **Pagos**.
2. Registrá o consultá el pago de una inscripción.

Nota. Si tu club no tiene un plan con `accesoDashboardClub`, no verás el panel de Club.

5. Administrador de Federación / Admin

5.1 Crear un evento

1. Entrá a **Eventos** → **Nuevo evento**.

2. Completá el formulario:

Campo	Descripción	Obligatorio
Nombre	Nombre del evento	Sí
Fecha / fechas	Fechas de competencia (editables luego)	Sí
Ubicación	Sede	Recomendado
Federación	Federación organizadora	Sí
Inscripciones abiertas	Habilita inscripciones de clubes	Sí

3. Guardá. El evento aparece en la grilla.

5.2 Configurar pruebas

1. Abrí el evento → **Configurar pruebas**.
2. Agregá cada prueba con **categoría, bote, distancia y sexo**.
3. Podés cargar **múltiples categorías**.
4. Ajustá los **horarios** en el cronograma. El cambio se propaga a las fases.
5. Guardá.

Nota. Existe un modo **"todo manual"** para eventos sin consolas de juez: habilita la carga manual de tiempos.

5.3 Siembra y generación de fases

1. Entrá a **Resultados** del EventoPrueba.
2. Presioná **Generar fases**: se crean las series con carriles sorteados.
3. Alternativamente, usá **Generar manual** si necesitás indicar colocaciones a mano.
4. Revisá las tarjetas de fase (FaseCard); podés editar horarios/carriles o eliminar una fase.
5. Consultá la **auditoría de progresión** para ver el detalle del armado.

5.4 Promover una etapa (ICF)

1. Con la etapa finalizada y resultados completos, presioná **Promover**.
2. El sistema aplica la regla ICF y asigna los clasificados a la etapa siguiente.
3. Verificá el resultado en las tarjetas de la etapa siguiente.

5.5 Habilitar operadores por evento

En **Logins** podés habilitar/deshabilitar qué roles pueden realizar controles en cada evento.

5.6 Planes y funcionalidades

Feature	Flag del plan	Qué habilita
Acceso a SportTrack	accesoSportTrack	Uso del sistema
Acceso a SIGDEF	accesoSigdef	Módulos federativos
Controles live	accesoControlesLive	Consolas de juez (solo tier L: planes 6 y 9)
Dashboard de Club	accesoDashboardClub	Panel de Club
Carga de imágenes	permitirCargaImagenes	Imágenes en eventos
Resultados en tiempo real	resultadosTiempoReal	Pizarra en vivo
Exportación PDF	exportacionPdf	Descarga de PDF
Máximo de atletas	maxAtletas	Límite de atletas (-1 = sin límite)

5.7 Exportar reportes

1. En el cronograma o resultados, presioná **Exportar**.
2. Elegí el tipo: cronograma completo, programa inicial, fase, grupo, prueba o start list.
3. El archivo PDF se descarga. También hay exportación **CSV**.

6. Largador

6.1 Prepararse

1. Entrá a `/jueces/largador`.
2. Elegí el **evento** y la **serie**.
3. Esperá a que el indicador de conexión muestre **Conectado**.

6.2 Dar la largada

1. Verificá que la serie esté lista.
2. Presioná **Largar**.
3. El sistema registra el **t0 sincronizado con el servidor** (no con tu reloj local).
4. El cronometrista recibe la señal y suena el **timbre de largada**.

Advertencia. Si la conexión se cae, el t0 se guarda y se reintenta automáticamente. **Nunca se recalcula** el instante de largada.

6.3 Reiniciar una regata

1. Con la regata en curso, abrí **Reiniciar regata**.
2. Elegí el motivo:

Categoría	Cuándo usarla
Mala largada / partida en falso	Salida incorrecta
Postergación o suspensión (clima, seguridad)	Condiciones externas
Problema técnico (cronometraje, sistema)	Falla del sistema
Problema externo imprevisto	Otros imprevistos
Otro motivo	Requiere detalle de al menos 10 caracteres

3. Confirmá el reinicio. Los conectados reciben la notificación y la serie vuelve a estar lista.

Aviso del sistema. Usá el reinicio **solo** por incidentes operativos. No corresponde para corregir siembras, pases de etapa ni el armado del cronograma.

7. Cronometrista / Finalizador

En la interfaz, este rol aparece como **"Finalizador"**.

7.1 Prepararse

1. Entrá a `/jueces/llegada`.
2. Elegí evento y serie.
3. Verificá la conexión y el estado del reloj sincronizado.

7.2 Registrar tiempos de llegada

1. Al sonar el **timbre de largada**, el sistema habilita la captura.
2. Por cada carril, presioná el botón correspondiente en el momento en que el bote cruza la meta.
3. La confirmación es inmediata (aunque el envío al servidor tarde).
4. Repetí para todos los carriles.

Consejo. Desbloquee el audio de tu dispositivo con un primer toque antes de la primera largada (políticas de reproducción automática de los navegadores).

7.3 Si la señal es mala (modo offline)

1. Seguí registrando los tiempos con normalidad: se guardan en la **cola local**.
2. Descargá el **PDF de respaldo** desde el botón de respaldo (válido 24 h).
3. Cuando vuelva la conexión, los tiempos se sincronizan solos.
4. Si algo quedó pendiente en el servidor, Soporte puede confirmarlo (ver sección 10).

Indicador	Significado	Acción
Conectado	Envío en tiempo real	Normal
Reintentando	Conexión inestable	Seguir capturando
Sin conexión	Cola local activa	Capturar y bajar respaldo PDF

8. Juez de Control

1. Entrá a `/juez-control`.
2. Elegí el evento y la serie que esté **En revisión**.
3. Revisá los tiempos y posiciones cargados.
4. Si todo está correcto, presioná **Oficializar**: los resultados pasan a definitivos y se publican en la pizarra pública.
5. Si hay errores, corregí los tiempos antes de oficializar.
6. Toda acción queda registrada en la auditoría.

Nota. Una vez oficializado, un cambio requiere reabrir la revisión desde la operación correspondiente.

9. Control Técnico

1. Entrá a `/control-tecnico`.
2. Supervisá la operación técnica de la competencia.
3. Según tu asignación por evento, podés acceder a consolas de largada y llegada (tu rol está incluido en los guards de `/jueces/largador` y `/jueces/llegada`).

10. SuperAdmin / Soporte

10.1 Administrar federaciones y clubes

1. Entrá a **Federaciones** y creá/editá federaciones.
2. En **Clubes**, gestioná clubes y su estado activo/inactivo.

10.2 Planes SaaS

1. Entrá a **SaaS**.
2. Asigná el plan a un club y consultá las métricas globales.

10.3 Logins y usuarios

1. Entrá a **Logins**.
2. Creá usuarios indicando el rol (`rolFederacion`).
3. El sistema valida el plan:
 - logins de **club** requieren `accesoDashboardClub`;
 - logins de **juez** requieren `accesoControlesLive`.
4. Un login de Club **debe** estar vinculado a un club.

10.4 Colas temporales de tiempos (soporte)

1. Entrá a **Soporte** → pestaña **Colas temporales**.
2. Verás las entradas pendientes (fase y usuario).
3. Para consolidar: seleccioná una entrada → **Confirmar**.
4. Para eliminar: **Descartar** y confirmá.

10.5 Auditoría

1. En **Soporte** podés ver los **logs de errores** y limpiarlos.
2. En **Actividad por evento** podés agrupar la actividad por evento y marcarlo como **"sin problemas"**.

10.6 Notificaciones y mensajería

1. Usá la **campana** para ver avisos por categoría.

2. En **Mensajes** gestioná hilos y comunicaciones masivas.

Nota. El usuario alias `soporte_tecnico` se comporta como SuperAdmin, aunque no tenga el rol literal.

11. FAQ y resolución de problemas

¿No puedo iniciar sesión?

- Verificá usuario y contraseña (distingue mayúsculas).
- Si tu sesión expiró, el sistema te redirige a `/login`; volvé a ingresar.
- Si el problema persiste, contactá a Soporte.

Estoy sin conexión durante el cronometraje

1. Seguí capturando tiempos: quedan en la cola local.
2. Descargá el **PDF de respaldo**.
3. Al volver la conexión, la app sincroniza automáticamente.
4. Si faltan tiempos en el servidor, Soporte los confirma desde las **Colas temporales**.

Quedaron tiempos pendientes

- Es normal si hubo un corte de red. Soporte puede **confirmar** o **descartar** la cola desde su panel.

¿Puedo reiniciar una regata?

- Sí, con motivo y categoría. Si elegís "Otro", el detalle debe tener 10+ caracteres.
- No uses el reinicio para corregir siembras ni cronograma.

No veo una función que otro usuario sí ve

- Puede ser por **rol** o por **plan SaaS**. Los controles live dependen del plan tier **L** (planes 6 y 9). Consultá a tu administrador.

¿Cómo exporto a PDF?

- Desde el cronograma/resultados usá **Exportar**. Si el botón no aparece, tu plan puede no incluir `exportacionPdf`.

El timbre de largada no suena

- Asegurate de haber hecho un primer toque en la pantalla (desbloqueo de audio).
- Subí el volumen del dispositivo.
- Solo suena para largadas "en vivo" (no al abrir una carrera antigua).

La pizarra en vivo no se actualiza

- Verificá tu conexión. La app reintenta automáticamente.
- Recargá la página si el problema persiste.

¿Funciona en el celular?

- Sí. La app puede instalarse como aplicación Android (`SportTrack Jueces`). En Android se usa solo token (sin cookie), por lo que el login es igual de seguro.

Pie del pack

Documento anterior	Documento actual	Documento siguiente
← 03 Diagramas de Flujo	04 — Manual de Usuario	05 — Manual Técnico →

[README](#) · [01](#) · [02](#) · [03](#) · [04](#) · [05](#) · [06](#)

05 — Manual Técnico

Proyecto: SportTrack — Frontend de Competencias **Última actualización:** 2026-09-27 **Audiencia:** desarrolladores, DevOps y soporte técnico

Índice

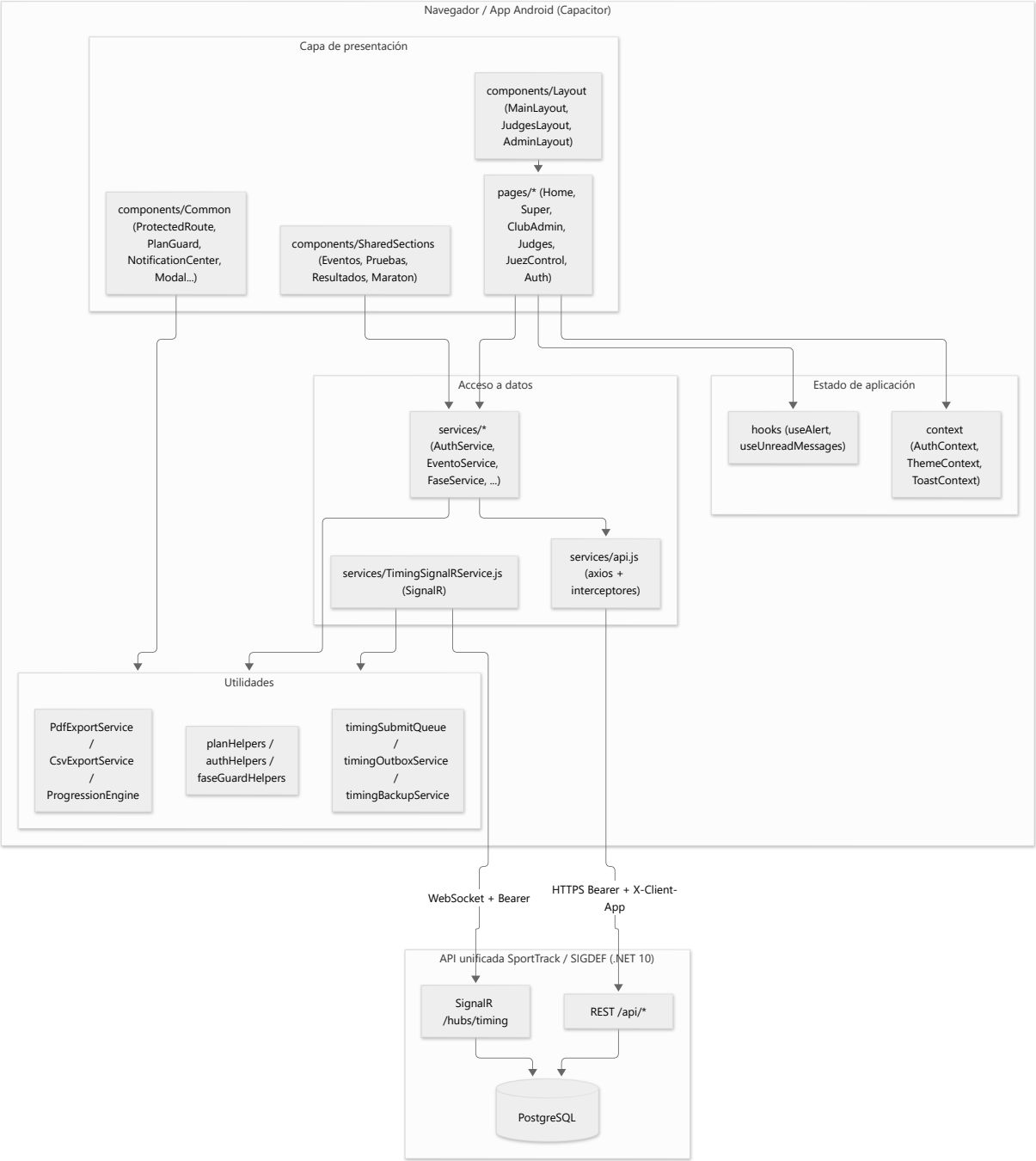
1. Introducción y alcance
 2. Arquitectura general
 3. Stack y versiones reales
 4. Estructura del código
 5. Módulos y servicios
 6. Contratos de API consumidos
 7. SignalR: métodos y eventos
 8. Estado y almacenamiento local
 9. Planes SaaS y guards
 10. Estrategia offline-first
 11. Seguridad
 12. Build y entornos
 13. Despliegue
 14. Mantenimiento y troubleshooting
 15. Deuda técnica
 16. Pie del pack
-

1. Introducción y alcance

Este manual describe la arquitectura, el stack, la organización del código, los contratos de integración y las prácticas de operación del frontend **SportTrack-Front**, verificado contra el repositorio. No documenta la implementación del backend (`SportTrack-Sigdef` , ASP.NET Core **.NET 10** + PostgreSQL).

2. Arquitectura general

La SPA se organiza en capas: **presentación (pages/components)**, **estado de aplicación (context/hooks)**, **acceso a datos (services)** y **utilidades puras (utils)**. Se comunica con la API unificada por REST (axios) y por WebSockets (SignalR).



2.1 Flujo de una petición

- 1. Un componente llama a un método de servicio (p. ej. `EventoService.getAll()`).
- 2. El servicio usa la instancia axios de `api.js`.
- 3. El interceptor de request inyecta `Authorization: Bearer <token>` y `X-Client-App: sporttrack`.
- 4. El interceptor de response normaliza errores (`getUserFacingError`) y, ante `401`, limpia el almacenamiento local.
- 5. El componente actualiza su estado y renderiza.

3. Stack y versiones reales

Verificado contra `package.json` (versión del proyecto `1.0.0`).

3.1 Runtime y framework

Componente	Paquete	Versión
Biblioteca UI	react / react-dom	^18.3.1

Componente	Paquete	Versión
Ruteo	<code>react-router-dom</code>	<code>^6.26.2</code>
Bundler / dev server	<code>vite</code>	<code>^5.4.10</code>
Plugin React para Vite	<code>@vitejs/plugin-react</code>	<code>^4.3.3</code>
Tipos	<code>@types/react</code> , <code>@types/react-dom</code>	<code>^18.3.x</code>

3.2 Datos, tiempo real y reportes

Componente	Paquete	Versión
HTTP	<code>axios</code>	<code>^1.7.7</code>
Tiempo real	<code>@microsoft/signalr</code>	<code>^8.0.7</code>
PDF	<code>jspdf</code>	<code>^4.2.1</code>
PDF (tablas)	<code>jspdf-autotable</code>	<code>^5.0.7</code>
Hashing	<code>bcryptjs</code>	<code>^3.0.3</code>

3.3 Maps y UI

Componente	Paquete	Versión
Mapas	<code>react-simple-maps</code>	<code>^3.0.0</code>
Geo	<code>d3-geo</code>	<code>^3.1.1</code>
TopoJSON	<code>topojson-client</code>	<code>^3.1.0</code>
Iconos	<code>lucide-react</code>	<code>^1.8.0</code>

3.4 Móvil (Capacitor)

Componente	Paquete	Versión
Core	<code>@capacitor/core</code>	<code>7.6.8</code>
Android	<code>@capacitor/android</code>	<code>7.6.8</code>
CLI	<code>@capacitor/cli</code>	<code>7.6.8</code>
Keyboard	<code>@capacitor/keyboard</code>	<code>^7.0.6</code>
Status Bar	<code>@capacitor/status-bar</code>	<code>^7.0.6</code>

3.5 Scripts de `package.json`

Script	Comando	Uso
<code>dev</code>	<code>vite</code>	Servidor de desarrollo
<code>build</code>	<code>vite build</code>	Build de producción web
<code>preview</code>	<code>vite preview</code>	Previsualización del build
<code>build:android</code>	<code>vite build --mode capacitor && npx cap sync android</code>	Build + sincronización Android
<code>open:android</code>	<code>npx cap open android</code>	Abrir el proyecto Android

4. Estructura del código

```
SportTrack-Front/
├── capacitor.config.json      # Config de la app Android (appId com.sporttrack.jueces)
├── vite.config.js            # Alias @, proxy /api y /hubs, base para capacitor
├── .env.example              # Plantilla de variables de entorno
├── index.html
├── src/
│   ├── assets/              # Recursos estáticos
│   ├── config/              # adminNavItems.jsx (navegación del panel admin)
│   ├── context/             # AuthContext, ThemeContext, ToastContext
│   ├── hooks/               # useAlert, useUnreadMessages
│   ├── components/
│   │   ├── Common/         # ProtectedRoute, PlanGuard, NotificationCenter, Modal,
│   │   │                   #   ConfirmDialog, ReiniciarFaseDialog, ToastContainer,
│   │   │                   #   WorldGlobe, EventAuditCards, ProgressionAudit...
│   │   ├── Layout/         # MainLayout, JudgesLayout, AdminLayout, Navbar, Sidebar, AppFooter
│   │   └── SharedSections/  # GestionEventosSection, ConfigurarPruebasModal, EventForm,
│   │                       #   PruebaForm, FaseCard, ResultadosTable, useResultados.js,
│   │                       #   ConfigurarMaratonModal, maraton/*
│   ├── pages/
│   │   ├── Home/           # Home, LiveResults, PlanDetails
│   │   ├── Auth/           # Login
│   │   ├── Super/          # Dashboard, AdminHome, sections/*
│   │   ├── ClubAdmin/      # Dashboard, sections/*
│   │   ├── Judges/         # JudgesDashboard, StarterDashboard, FinisherDashboard, ManualTiming
│   │   ├── JuezControl/    # JuezControlDashboard
│   │   └── Shared/         # Mensajes, CampanaDetalle, DestinatariosMultiSelect
│   ├── services/           # 30 servicios (ver §5)
│   └── utils/              # 32 utilidades (helpers de plan, auth, timing, PDF, etc.)
```

4.1 Puntos de entrada clave

Archivo	Rol
src/main.jsx	Bootstrap de React, providers globales.
src/App.jsx	Definición de rutas y guards.
src/services/api.js	Cliente axios y interceptores.
src/utils/constants.js	API_BASE_URL , ENDPOINTS , STORAGE_KEYS , APP_NAME .

5. Módulos y servicios

5.1 Servicios HTTP (src/services/)

Servicio	Responsabilidad
api.js	Instancia axios, interceptores de request/response, X-Client-App , manejo de 401.
AuthService	Login, registro, usuarios, perfil, logout, validateSession (/auth/me).
EventoService	CRUD de eventos y consulta por club/federación.
FaseService	Fases: generar, generar manual, promover, iniciar, finalizar, reiniciar, revisión, batch update, auditoría de progresión.
ResultadoService	Batch update de resultados y resultados por fase.
InscripcionService	Alta, edición, borrado y toggle de siembra de inscripciones.
ParticipanteService	Alta/edición de participantes (atletas).
ClubService	CRUD de clubes.
FederacionService	Listado/consulta de federaciones (con normalización).

Servicio	Responsabilidad
ConfigService	Pruebas, categorías, botes, distancias y asignación de pruebas a eventos.
SaaSService	Planes, asignar plan, status de clubes, toggle activo, métricas, federaciones.
PagoService	Historial de pagos, registro y toggles de <code>alDia / pagado</code> .
MessageService	Hilos, campañas, responder, marcar leído, contador de no leídos.
AuditoriaService	Logs de auditoría, por eventos, acciones de cliente.
SupportService	Logs de soporte y colas temporales de tiempos (outbox).
BackupService	Descarga e historial de backups.
TelemetryService	Envío de errores de frontend.
AudienceService	Métricas de audiencia en vivo, picos y capacidad.
SignalRService	Obsoleto (<code>@deprecated</code>); conservado por compatibilidad. Usar <code>TimingSignalRService</code> .
TimingSignalRService	Conexión SignalR de tiempos, sincronización de reloj, envío de tiempos y largadas.
timingSubmitQueue	Cola local de envío de tiempos con <code>timingBackupService</code> y <code>auditActionQueue</code> .
timingOutboxService	Cliente del outbox de tiempos en el servidor.
timingBackupService	Persistencia del respaldo local de tiempos (TTL 24 h).
timingAuditRepair	Reparación/registro de auditoría de timing.
auditActionQueue / auditActionTracker	Encolado y seguimiento de acciones de auditoría.
PdfExportService	Exportación PDF (fase, grupo, prueba, start list maratón, cronogramas, programa, respaldo).
CsvExportService	Exportación CSV.
SchedulerService	Motor/armado de cronograma.

5.2 Utilidades destacadas (`src/utils/`)

Utilidad	Responsabilidad
<code>constants.js</code>	<code>ENDPOINTS</code> , <code>STORAGE_KEYS</code> , <code>API_BASE_URL</code> , <code>PUBLIC_APP_URL</code> , <code>liveResultsUrl</code> .
<code>planHelpers.js</code>	Normalización y flags de planes SaaS; <code>canAccess*</code> .
<code>authHelpers.js</code>	Rol, normalización de usuario, alias <code>soporte_tecnico</code> , ruta por rol, búsqueda.
<code>faseGuardHelpers.js</code>	Guardas de fases por rol/estado.
<code>controlTecnico.js</code>	Lógica de permisos de control técnico.
<code>ProgressionEngine.js</code>	Construcción de la traza de progresión de cada atleta.
<code>promotionHelpers.js</code>	Apoyo a la promoción ICF.
<code>resultadosHelpers.js</code>	Cálculo de posiciones por fase.
<code>raceTimeUtils.js</code>	Formateo y conversión de tiempos de carrera.
<code>timingMath.js</code>	Sincronización de reloj y largada (<code>getSyncedNow</code> , <code>pendingRaceStart</code>).
<code>raceStartBell.js</code>	Timbre de largada vía Web Audio API.
<code>reiniciarFaseConstants.js</code>	Categorías, aviso y validación del reinicio de regata.
<code>maratonScheduleUtils.js</code> / <code>maratonStartListUtils.js</code>	Horarios y armado de start list de maratón.
<code>notificationHelpers.js</code>	Metadatos, agrupación y mapeo de notificaciones.
<code>apiHelpers.js</code>	Normalización de respuestas de API.
<code>userFacingError.js</code>	Traducción de errores a mensajes de usuario.
<code>tokenUtils.js</code>	Decodificación JWT y verificación de expiración.
<code>deviceUtils.js</code>	Parseo de User-Agent para la auditoría de dispositivo.

6. Contratos de API consumidos

Todas las rutas se resuelven sobre `API_BASE_URL` (`/api`). La columna **Rol** indica el actor que típicamente consume el endpoint en el frontend.

6.1 Autenticación (AuthService)

Método	Ruta	Rol	Descripción
POST	<code>/auth/login</code>	Público	Inicio de sesión; devuelve token + usuario + plan.
POST	<code>/auth/register</code>	Admin/SuperAdmin	Alta de usuario.
GET	<code>/auth/me</code>	Autenticado	Validación de sesión.
GET	<code>/auth/usuarios</code>	Admin/SuperAdmin	Listado de usuarios.
PUT	<code>/auth/usuarios/{id}/password</code>	Admin/SuperAdmin	Cambio de contraseña.
PUT	<code>/auth/usuarios/{id}/perfil</code>	Autenticado	Edición de perfil.
PATCH	<code>/auth/usuarios/{id}/toggle-activo</code>	Admin/SuperAdmin	Activar/desactivar usuario.
DELETE	<code>/auth/usuarios/{id}</code>	Admin/SuperAdmin	Eliminar usuario.
POST	<code>/auth/logout</code>	Autenticado	Cierre de sesión.

6.2 Eventos (EventoService)

Método	Ruta	Rol	Descripción
GET	<code>/eventos?clubId=&federacionId=</code>	Autenticado	Listado de eventos por alcance.
GET	<code>/eventos/proximos?...</code>	Autenticado/Público	Próximos eventos.
POST	<code>/eventos</code>	Admin/SuperAdmin	Crear evento.
GET	<code>/eventos/{id}</code>	Autenticado/Público	Detalle de evento.
PUT	<code>/eventos/{id}</code>	Admin/SuperAdmin	Editar evento.
DELETE	<code>/eventos/{id}</code>	Admin/SuperAdmin	Eliminar evento.
PUT	<code>/eventos/pruebas/{eventoPruebaId}</code>	Admin/SuperAdmin	Editar una prueba asignada.

6.3 Pruebas y catálogos (ConfigService)

Método	Ruta	Rol	Descripción
GET	<code>/eventos/{eventoId}/pruebas</code>	Autenticado	Pruebas de un evento.
POST	<code>/eventos/{eventoId}/pruebas</code>	Admin/SuperAdmin	Asignar prueba a evento.
POST	<code>/eventos/{eventoId}/pruebas/largada</code>	Admin/SuperAdmin	Prueba de largada.
PUT	<code>/eventos/pruebas/{assignId}</code>	Admin/SuperAdmin	Editar prueba asignada.
DELETE	<code>/eventos/pruebas/{assignId}</code>	Admin/SuperAdmin	Quitar prueba.
GET	<code>/pruebas</code>	Autenticado	Catálogo de pruebas.
POST	<code>/pruebas</code>	Admin/SuperAdmin	Crear prueba.
GET	<code>/botes</code> , <code>/categorias</code> , <code>/distancias</code>	Autenticado	Catálogos maestros.

6.4 Inscripciones (InscripcionService)

Método	Ruta	Rol	Descripción
POST	<code>/inscripciones</code>	Club/Admin	Crear inscripción.
POST	<code>/inscripciones/registro</code>	Club/Admin	Registro de inscripción.
GET	<code>/inscripciones/evento-prueba/{id}</code>	Autenticado	Inscripciones por EventoPrueba.
GET	<code>/inscripciones/evento/{eventoId}/club/{clubId}</code>	Club/Admin	Inscripciones por evento y club.
PUT	<code>/inscripciones/{id}</code>	Club/Admin	Editar inscripción.
DELETE	<code>/inscripciones/{id}</code>	Club/Admin	Eliminar inscripción.
PATCH	<code>/inscripciones/{id}/toggle-seeding</code>	Admin	Habilitar/deshabilitar siembra.

6.5 Fases (FaseService)

Método	Ruta	Rol	Descripción
GET	/fases/EventoPrueba/{id}	Admin/Operadores	Fases de un EventoPrueba.
POST	/fases/Generar/{id}	Admin/SuperAdmin	Generar fases automáticamente.
POST	/fases/GenerarManual/{id}	Admin/SuperAdmin	Generar fases manualmente.
POST	/fases/GenerarLargadaMaraton	Admin/SuperAdmin	Generar largada de maratón.
POST	/fases/Promover/{id}	Admin/SuperAdmin	Promover etapa ICF.
GET	/fases/ProgresionAudit/{id}	Admin/SuperAdmin	Auditoría de progresión.
POST	/fases/BatchUpdate	Admin/SuperAdmin	Actualización masiva de fases.
PUT	/fases/{id}/details	Admin/SuperAdmin	Editar detalles de fase.
POST	/fases/{id}/Iniciar?startTime=	Largador/Admin	Iniciar regata (t0).
POST	/fases/{id}/Finalizar	Cronometrista/Admin	Finalizar regata.
POST	/fases/{id}/Reiniciar	Largador/Admin	Reiniciar regata (motivo, categoría).
POST	/fases/{id}/EnviarARevision	Cronometrista/Admin	Enviar a revisión.
DELETE	/fases/{id}	Admin/SuperAdmin	Eliminar fase.
GET	/eventos/{id}/fases	Autenticado	Fases de un evento.

6.6 Resultados (ResultadoService)

Método	Ruta	Rol	Descripción
PUT	/resultados/BatchUpdate	JuezControl/Admin	Actualización masiva de resultados.
GET	/resultados/Fase/{id}	Autenticado/Público	Resultados de una fase.

6.7 Outbox de tiempos (timingOutboxService)

Método	Ruta	Rol	Descripción
POST	/timing-outbox	Cronometrista	Upsert de entrada pendiente.
GET	/timing-outbox/pending	Cronometrista/Soporte	Listar pendientes.
POST	/timing-outbox/flush	Cronometrista	Reintentar envío.
POST	/timing-outbox/{faseId}/commit	Cronometrista/Soporte	Consolidar tiempos.
DELETE	/timing-outbox/{faseId}	Cronometrista/Soporte	Eliminar entrada.

6.8 SaaS y federaciones (SaaSService)

Método	Ruta	Rol	Descripción
GET	/SaaS/planes	Admin/SuperAdmin	Planes disponibles.
POST	/SaaS/asignar-plan?clubId=&planId=	SuperAdmin	Asignar plan a club.
GET	/SaaS/clubes-status	SuperAdmin	Estado de clubes.
PATCH	/SaaS/clubes/{clubId}/toggle-activo	SuperAdmin	Activar/desactivar club.
POST	/SaaS/create-federacion	SuperAdmin	Crear federación.
PUT	/Federaciones/{id}	SuperAdmin	Editar federación.
DELETE	/Federaciones/{id}	SuperAdmin	Eliminar federación.
GET	/SaaS/global-metrics	SuperAdmin	Métricas globales.
GET	/federaciones , /federaciones/{id}	Autenticado	Consulta de federaciones.

6.9 Clubes, participantes y pagos

Método	Ruta	Rol	Descripción
GET	/clubes , /clubes/{id}	Autenticado	Clubes.
POST/PUT/DELETE	/clubes · /clubes/{id}	Admin/SuperAdmin	Gestión de clubes.

Método	Ruta	Rol	Descripción
GET/PUT/DELETE	/participantes/{id}	Club/Admin	Gestión de participantes.
GET	/participantes/club/{clubId}	Club/Admin	Atletas por club.
GET	/pagos/historial	Admin/SuperAdmin	Historial de pagos.
POST	/pagos/registrar	Club/Admin	Registrar pago.
PUT	/pagos/clubes/{clubId}/toggle	Admin	Estado de pago del club.
PUT	/pagos/atletas/{atletaId}/toggle	Admin	Estado de pago del atleta.
PUT	/pagos/inscripciones/{id}/toggle	Admin	Estado de pago de inscripción.

6.10 Mensajería (MessageService)

Método	Ruta	Rol	Descripción
GET	/mensajes/hilos	Autenticado	Listar hilos.
GET	/mensajes/hilos/{id}	Autenticado	Detalle de hilo.
POST	/mensajes/hilos	Autenticado	Crear hilo.
POST	/mensajes/hilos/masivo	Admin/SuperAdmin	Envío masivo.
GET	/mensajes/campanas , /mensajes/campanas/{id}	Admin/SuperAdmin	Campañas.
POST	/mensajes/hilos/{id}/responder	Autenticado	Responder.
PATCH	/mensajes/hilos/{id}/leer	Autenticado	Marcar leído.
GET	/mensajes/no-leidos/count	Autenticado	Contador de no leídos.

6.11 Auditoría y soporte

Método	Ruta	Rol	Descripción
GET	/Auditoria	SuperAdmin/Soporte	Logs de auditoría.
GET	/Auditoria/por-eventos	SuperAdmin/Soporte	Auditoría agrupada por evento.
POST	/Auditoria/client-action	Autenticado	Registrar acción de cliente.
GET	/Auditoria/{id}	SuperAdmin/Soporte	Detalle de log.
POST	/Auditoria/por-evento/{eventoId}/sin-problemas	SuperAdmin/Soporte	Marcar evento sin problemas.
GET	/support/logs	SuperAdmin/Soporte	Logs de soporte.
DELETE	/support/logs/clear	SuperAdmin/Soporte	Limpiar logs.
GET	/support/timing-outbox	SuperAdmin/Soporte	Colas temporales.
POST	/support/timing-outbox/{faseId}/commit	SuperAdmin/Soporte	Confirmar cola.
DELETE	/support/timing-outbox/{id}	SuperAdmin/Soporte	Descartar cola.
POST	/support/frontend-error	Autenticado	Telemetría de errores.
POST	/support/client-action	Autenticado	Registro de acción de cliente.

6.12 Backups, audiencia y utilidades

Método	Ruta	Rol	Descripción
GET	/backup/download	SuperAdmin	Descargar backup.
GET	/backup/history?limit=	SuperAdmin	Historial de backups.
GET	/Audience/live	SuperAdmin	Audiencia en vivo.
GET	/Audience/peaks?limit=	SuperAdmin	Picos de audiencia.
GET	/Audience/capacity	SuperAdmin	Capacidad.
PUT	/Audience/capacity	SuperAdmin	Ajustar capacidad.

7. SignalR: métodos y eventos

Hub: `/hubs/timing` (resuelto por `resolveHubUrl()` desde `VITE_SIGNALR_HUB_URL`, con fallback a `{API_BASE_URL sin /api}/hubs/timing`). **Autenticación:** `accessTokenFactory` inyecta el JWT almacenado. **Reconexión:** `withAutomaticReconnect()`; al reconectar se re-suscriben los grupos y se sincroniza el reloj.

7.1 Métodos cliente → servidor

Método	Uso en el frontend
<code>JoinEventGroup(eventoId, userName, role)</code>	Suscripción a un evento.
<code>JoinRaceGroup(faseId, userName, role)</code>	Suscripción a una regata.
<code>LeaveRaceGroup(faseId)</code>	Salir del grupo de regata.
<code>JoinUserNotificationsGroup()</code>	Notificaciones del usuario.
<code>JoinFederationNotificationsGroup(federacionId)</code>	Notificaciones de federación.
<code>JoinOperatorsGroup()</code>	Notificaciones de operadores (admin/jueces/club).
<code>GetServerTime()</code>	Sincronización de reloj (5 muestras).
<code>RequestStartRace(faseId, t0Iso)</code>	Largada de regata.
<code>RequestResetRace(faseId, motivo, categoria)</code>	Reinicio de regata.
<code>SendTime(faseId, resultadoId, timeStr, ms)</code>	Envío de tiempo de llegada.
<code>UpdateResultStatus(faseId, resultadoId, status)</code>	Cambio de estado de resultado.
<code>RequestPaymentStatusChange(clubNombre, clubId)</code>	Solicitud de cambio de estado de pago.

7.2 Eventos servidor → cliente

Evento	Payload	Uso
<code>RaceStarted</code>	<code>(id, serverTime)</code>	Largada de la regata suscrita.
<code>RaceReset</code>	<code>(id)</code>	Regata reiniciada.
<code>RaceFinished</code>	<code>(id)</code>	Regata finalizada.
<code>RaceInReview</code>	<code>(id)</code>	Regata en revisión.
<code>TimeReceived</code>	<code>(resultadoId, timeStr, ms)</code>	Tiempo recibido.
<code>LapRecorded</code>	<code>(resultadoId, time)</code>	Vuelta/parcial registrado.
<code>ResultadoActualizado</code>	<code>(eventoPruebaId, resultado)</code>	Resultado actualizado.
<code>RacePresenceUpdated</code>	<code>presenceList</code>	Presencia en la regata.
<code>EventPresenceUpdated</code>	<code>presenceList</code>	Presencia en el evento.
<code>globalRaceStarted</code>	<code>(faseId, serverTime)</code>	Largada global (pizarra).
<code>globalRaceInReview</code>	<code>(fase)</code>	Revisión global.
<code>globalRaceOfficialized</code>	<code>(faseId)</code>	Oficialización global.
<code>globalTimeReceived</code>	<code>(faseId, resultadoId, timeStr, ms)</code>	Tiempo global.
<code>GlobalResultStatusUpdated</code>	<code>(resId, status)</code>	Estado de resultado global.
<code>paymentStatusChangeRequested</code>	<code>data</code>	Solicitud de cambio de pago.
<code>newMessageReceived</code>	<code>payload</code>	Nuevo mensaje.
<code>newEventCreated</code>	<code>payload</code>	Nuevo evento.

El servicio registra **alias en minúscula** para eventos que el backend puede emitir con distinta capitalización (p. ej. `raceinreview`, `globaltimereceived`).

7.3 Sincronización de reloj

- Se toman 5 muestras de `GetServerTime`; se descartan `RTT > 1500 ms`.
- Se calcula el offset por la **mediana** de las muestras y se expone `getClockSyncStatus()`.
- El `t0` de largada se obtiene con `getSyncedNow()` (hora local + offset), garantizando consistencia.

8. Estado y almacenamiento local

8.1 Claves de `localStorage` (`STORAGE_KEYS`)

Constante	Clave	Contenido
<code>AUTH_TOKEN</code>	<code>sporttrack_auth_token</code>	JWT de acceso.
<code>REFRESH_TOKEN</code>	<code>sporttrack_refresh_token</code>	Refresh token.
<code>USER_DATA</code>	<code>sporttrack_user_data</code>	Datos del usuario y su plan.
<code>THEME</code>	<code>sporttrack_theme</code>	Tema (claro/oscuro).
<code>SIDEBAR_PINNED</code>	<code>sporttrack_sidebar_pinned</code>	Estado de la barra lateral.
<code>PENDING_AUDIT_ACTIONS</code>	<code>sporttrack_pending_audit_actions</code>	Cola de acciones de auditoría.

8.2 Otras claves usadas

Clave	Uso
<code>theme</code>	Preferencia de tema (ThemeContext/TemaToggle).
<code>results_selected_evento</code>	Evento seleccionado en resultados.
<code>results_selected_prueba</code>	Prueba seleccionada en resultados.
<code>locked_pruebas / sealed_pruebas</code>	Pruebas bloqueadas/selladas.
<code>finisher_event_id / finisher_fase_id</code>	Último evento/fase de la consola del finalizador.
<code>starter_event_id / starter_fase_id</code>	Último evento/fase de la consola del largador.
<code>sporttrack_pending_timing_saves</code>	Cola local de tiempos pendientes.
<code>sporttrack_dismissed_event_notifs</code>	Notificaciones de evento descartadas.
<code>sporttrack_pending_hilo_id</code> (sessionStorage)	Hilo de mensaje a abrir.

No hay dependencias de estado global externas (Redux, Zustand). El estado se maneja con Context API y estado local de componentes.

9. Planes SaaS y guards

9.1 Planes por ID (`planHelpers.js`)

Familia	IDs	Acceso
SIGDEF	1, 2, 3	<code>accesoSigdef</code>
SportTrack	4, 5, 6	<code>accesoSportTrack</code>
Pack Dúo	7, 8, 9	<code>accesoSigdef + accesoSportTrack</code>

9.2 Flags por plan

ID	Nombre	DashboardClub	Imágenes	TiempoReal	PDF	maxAtletas	ControlesLive
1	SIGDEF S	No	No	No	Sí	200	No
2	SIGDEF M	Sí	No	No	Sí	400	No
3	SIGDEF L	Sí	Sí	No	Sí	∞ (-1)	No
4	SportTrack S	No	No	No	Sí	200	No
5	SportTrack M	No	No	Sí	Sí	400	No
6	SportTrack L	No	No	Sí	Sí	∞ (-1)	Sí
7	Pack Dúo S	No	No	No	Sí	200	No
8	Pack Dúo M	Sí	No	Sí	Sí	400	No
9	Pack Dúo L	Sí	Sí	Sí	Sí	∞ (-1)	Sí

`accesoControlesLive` solo en tier **L** (IDs 6 y 9). `normalizePlan` acepta flags explícitos de la API y, si faltan, los deriva por ID y por nombre.

9.3 Guards

Guard	Ubicación	Función
<code>ProtectedRoute</code>	<code>components/Common/ProtectedRoute.jsx</code>	Exige autenticación; evalúa <code>requiredRole</code> y <code>requiereControlesLive</code> ; delega en <code>PlanGuard</code> .
<code>PlanGuard</code>	<code>components/Common/PlanGuard.jsx</code>	Bloquea por plan; pantalla "Función exclusiva del Ecosistema".
<code>faseGuardHelpers</code> / <code>controlTecnico</code>	<code>utils/</code>	Guardas finas de operación por rol/estado.

9.4 Matriz ruta ↔ guard

Ruta	Rol requerido	<code>requiereControlesLive</code>
<code>/login</code>	Público	—
<code>/</code>	Público	—
<code>/planes/:id</code>	Público	—
<code>/resultados/:id</code>	Público	—
<code>/club/*</code>	Club	No
<code>/super/*</code> , <code>/admin/*</code>	Admin, SuperAdmin	No
<code>/juez-control/*</code>	Admin, SuperAdmin, JuezControl	Sí
<code>/control-tecnico</code>	Admin, SuperAdmin, ControlTecnico	Sí
<code>/jueces</code>	Admin, SuperAdmin, Largador, Cronometrista, JuezControl, ControlTecnico	Sí
<code>/jueces/largador</code>	Admin, SuperAdmin, Largador, ControlTecnico	Sí
<code>/jueces/llegada</code>	Admin, SuperAdmin, Cronometrista, ControlTecnico	Sí
<code>/jueces/carga-manual</code>	Admin, SuperAdmin	No

10. Estrategia offline-first

10.1 Capas de resiliencia

Capa	Componente	Función
1. Cola local de tiempos	<code>timingSubmitQueue</code> + <code>sporttrack_pending_timing_saves</code>	Guarda tiempos capturados sin conexión.
2. Respaldo local	<code>timingBackupService</code> (TTL 24 h)	Persiste un respaldo reconstruible por fase.
3. Respaldo PDF	<code>PdfExportService.exportCronometristaRespaldo</code>	Permite recuperación manual.
4. Outbox servidor	<code>timingOutboxService</code> → <code>/timing-outbox/*</code>	Cola en servidor para commit/discard por Soporte.
5. Largada persistente	<code>timingMath.savePendingRaceStart</code>	t0 encolado y reintentado cada 4 s; nunca se regenera .
6. Reintentos HTTP	<code>retryWithBackoff</code>	Backoff para operaciones transitorias.

10.2 Garantías

- **Idempotencia del t0:** el instante de largada se captura una sola vez y se transporta por SignalR → HTTP → cola, sin recálculo (RN-09).
- **No pérdida de tiempos:** toda captura se persiste localmente antes de intentar el envío.
- **Sesión extendida:** token de 24 h (`tokenUtils`) para turnos largos.

11. Seguridad

11.1 Autenticación híbrida

- **Bearer en localStorage** : fuente de verdad para la API y SignalR.
- **Cookie X-Access-Token (HttpOnly)**: complemento seteado por el backend.
- **Motivo**: en el despliegue cross-origin (Vercel → Render) las cookies third-party suelen bloquearse; el Bearer asegura la operación. En **Capacitor** (`isNativePlatform()`) se desactiva `withCredentials` para evitar el fallo de login por CORS.

11.2 Encabezados y comportamiento del cliente

Elemento	Valor	Implementación
Token	<code>Authorization: Bearer <jwt></code>	Interceptor de request.
Cliente	<code>X-Client-App: sporttrack</code>	Config base + interceptor.
Credenciales	<code>withCredentials: true</code> (web)	Config de axios.
Timeout	30 s web / 45 s nativo	Config de axios.
Manejo de 401	Limpia <code>USER_DATA</code> y <code>AUTH_TOKEN</code>	Interceptor de response.

11.3 Autorización

- Doble barrera: **rol** (`ProtectedRoute`) y **plan** (`PlanGuard`).
- `SuperAdmin` y el alias `soporte_tecnico` omiten las restricciones de plan.
- La UI filtra acciones no permitidas (p. ej. no ofrece crear logins si el plan no lo habilita).

12. Build y entornos

12.1 Variables de entorno

Variable	Descripción	Default en código
<code>VITE_API_URL</code>	Base de la API (incluye <code>/api</code>)	<code>/api</code>
<code>VITE_SIGNALR_HUB_URL</code>	URL del hub de tiempos	derivada de la API
<code>VITE_PUBLIC_APP_URL</code>	URL pública del front (links live)	<code>https://sporttrack.pro</code>
<code>VITE_API_TARGET</code>	Destino del proxy de Vite (solo dev)	<code>https://sporttrack-sigdef.onrender.com</code>

- Los archivos `.env*` **no se versionan** (`.gitignore`); `.env.example` es la plantilla.
- En Vercel se cargan en **Settings** → **Environment Variables**.

12.2 Configuración de Vite

Aspecto	Valor
Alias	<code>@ → ./src</code>
base	<code>/</code> en web; <code>./</code> en modo <code>capacitor</code>
Puerto de dev	<code>5174</code> (<code>strictPort: true</code>)
host	<code>true</code> (accesible desde la red local)
Proxy <code>/api</code>	→ <code>VITE_API_TARGET</code>
Proxy <code>/hubs</code>	→ <code>VITE_API_TARGET</code> con <code>ws: true</code>

12.3 Configuración de Capacitor

Clave	Valor
<code>appId</code>	<code>com.sporttrack.jueces</code>
<code>appName</code>	SportTrack Jueces
<code>webDir</code>	<code>dist</code>
<code>androidScheme</code>	<code>https</code>
<code>allowNavigation</code>	<code>https://sporttrack-sigdef.onrender.com/*</code>

Clave	Valor
Plugins	StatusBar (DARK), Keyboard (native/DARK), CapacitorHttp (enabled)

13. Despliegue

13.1 Web (Vercel)

1. Conectar el repositorio `SportTrack-Front` a Vercel.
2. Configurar build `npm run build` y salida `dist`.
3. Definir `VITE_API_URL`, `VITE_SIGNALR_HUB_URL` y `VITE_PUBLIC_APP_URL` en Environment Variables.
4. Deploy. La SPA se sirve con `base: '/'`.

13.2 Android (Capacitor)

1. `npm run build:android` (build en modo `capacitor` + `cap sync`).
2. `npm run open:android` para abrir Android Studio.
3. Generar el APK/AAB.
4. Verificar que `allowNavigation` / `CapacitorHttp` permitan el acceso HTTPS al backend.

El detalle paso a paso está en el *documento 06*.

14. Mantenimiento y troubleshooting

Síntoma	Causa probable	Acción
401 recurrente con token válido	Desalineación Bearer/cookie, token vencido	Re-login; verificar prioridad del header Bearer en el backend.
SignalR no conecta	Hub caído o CORS	Verificar <code>/hubs/timing</code> y el proxy/ <code>VITE_SIGNALR_HUB_URL</code> .
Tiempos no llegan al servidor	Corte de red	Revisar cola local y Colas temporales en Soporte; confirmar outbox.
Largada no se registra	Sin conexión	El t0 se encola y reintenta; no regenerar manualmente.
Pizarra no actualiza	Suscripción caída	La reconexión automática re-suscribe; recargar si persiste.
El timbre no suena	Autoplay bloqueado	Requiere gesto previo del usuario.
Función oculta o bloqueada	Plan/rol insuficiente	Verificar flags de plan y <code>requiredRole</code> .
Error de import <code>/hubs/results</code>	Uso de <code>SignalRService</code> obsoleto	Migrar a <code>TimingSignalRService</code> .

14.1 Logs y observabilidad

- `TelemetryService` envía errores de frontend a `/support/frontend-error`.
- `AuditoriaService` + `auditActionQueue` registran actividad y acciones de cliente.
- `SoporteSection` centraliza logs, actividad por evento y colas temporales.

15. Deuda técnica

#	Ítem	Impacto	Recomendación
DT-01	<code>SignalRService.js</code> marcado <code>@deprecated</code> .	Confusión; hub inexistente.	Eliminar cuando no queden referencias.
DT-02	Sin suite de tests automatizados.	Regresiones no detectadas.	Incorporar Vitest + Testing Library.
DT-03	Sin TypeScript (solo <code>.jsx</code>).	Menor seguridad de tipos.	Migración gradual con JSDoc o TS.
DT-04	Uso intensivo de <code>localStorage</code> sin versionado.	Datos huérfanos ante cambios de esquema.	Versionar claves y migrar.

#	Ítem	Impacto	Recomendación
DT-05	<code>bcryptjs</code> en el bundle de frontend.	Peso innecesario; riesgo si se usa para hashing cliente.	Verificar uso y excluir del bundle de producción.
DT-06	Duplicación de <code>STORAGE_KEYS</code> parcial fuera de <code>constants.js</code> (claves de timing/jueces).	Inconsistencias.	Centralizar todas las claves.
DT-07	Contrato <code>Cronometrista</code> ↔ etiqueta "Finalizador".	Fricción de nomenclatura.	Documentar o unificar.
DT-08	Plan "Bronce" referenciado en <code>GestionResultadosSection</code> / <code>PlanDetails</code> pero ausente en <code>planHelpers</code> .	Inconsistencia funcional.	Alinear catálogo de planes o marcar como legado.

FUERA DE ALCANCE ACTUAL: la suite de tests, la migración a TypeScript y la unificación del catálogo de planes no están implementadas en el frontend a la fecha.

Pie del pack

Documento anterior	Documento actual	Documento siguiente
← 04 Manual de Usuario	05 — Manual Técnico	06 — Manual de Instalación y Despliegue →

[README](#) · [01](#) · [02](#) · [03](#) · [04](#) · [05](#) · [06](#)

06 — Manual de Instalación y Despliegue

Proyecto: SportTrack — Frontend de Competencias **Última actualización:** 2026-09-27 **Audiencia:** desarrolladores, DevOps y responsables de puesta en marcha

Índice

- 1. Requisitos previos
- 2. Obtención del código
- 3. Instalación de dependencias
- 4. Configuración del entorno
- 5. Ejecución local
- 6. Build de producción
- 7. Despliegue web (Vercel)
- 8. Build Android con Capacitor
- 9. Verificación y checklist de entrega
- 10. Resolución de problemas de instalación
- 11. Pie del pack

1. Requisitos previos

1.1 Software requerido

Requisito	Versión	Notas
Node.js	18 LTS o superior (recomendado 20 LTS)	Requerido por Vite 5.
npm	9+	Incluido con Node.
Git	Reciente	Para clonar el repositorio.
Navegador	Chrome/Edge/Firefox actualizado	La app usa Web Audio y WebSocket.

1.2 Requisitos opcionales

Requisito	Uso
Java JDK 17 + Android Studio	Necesarios solo para el build Android (Capacitor).
Android SDK	Para compilar el APK/AAB.

1.3 Acceso a servicios

Servicio	Necesario para
API SportTrack / SIGDEF	Consumir datos (https://sporttrack-sigdef.onrender.com/api).
Hub SignalR	Resultados en vivo (https://sporttrack-sigdef.onrender.com/hubs/timing).
Cuenta Vercel	Despliegue web.
Cuenta SuperAdmin	Validar funciones de administración.

2. Obtención del código

```
git clone <URL-del-repositorio-SportTrack-Front>
cd SportTrack-Front
```

Ruta local de referencia: c:\Users\EZEQU\source\reposFront\SportTrack-Front .

3. Instalación de dependencias

```
npm install
```

Esto instala las dependencias declaradas en `package.json` (React, Vite, SignalR, axios, jsPDF, Capacitor, etc.).

4. Configuración del entorno

4.1 Crear el archivo de entorno

Copía la plantilla según el entorno:

```
# Desarrollo web
cp .env.example .env.development

# Producción (build / Vercel)
cp .env.example .env.production

# Build Android (Capacitor)
cp .env.example .env.capacitor
```

4.2 Variables de entorno

Variable	Descripción	Ejemplo (dev)	Ejemplo (prod)
VITE_API_URL	URL base de la API, incluye <code>/api</code>	<code>http://localhost:5029/api</code>	<code>https://sporttrack-sigdef.onrender.com/api</code>
VITE_SIGNALR_HUB_URL	Hub SignalR de tiempos	<code>http://localhost:5029/hubs/timing</code>	<code>https://sporttrack-sigdef.onrender.com/hubs/timing</code>
VITE_PUBLIC_APP_URL	URL pública de la app (links de live)	<code>http://localhost:5174</code>	<code>https://<tu-dominio></code>
VITE_API_TARGET	Destino del proxy de Vite (solo dev)	<code>https://sporttrack-sigdef.onrender.com</code>	—

⚠ **Los archivos `.env*` no se versionan.** En **Vercel** se cargan en *Settings* → *Environment Variables*. Nunca subas credenciales reales al repositorio.

4.3 Proxy de desarrollo

`vite.config.js` define un proxy que redirige `/api` y `/hubs` (con `ws: true`) hacia `VITE_API_TARGET`. Esto permite trabajar con rutas relativas sin problemas de CORS en desarrollo.

5. Ejecución local

5.1 Modo desarrollo

```
npm run dev
```

- La app queda disponible en `http://localhost:5174` (configurado con `strictPort: true`).
- Si el puerto 5174 está ocupado, Vite **fallará** (no cambia de puerto). Liberá el puerto o ajustá `server.port`.
- El `host: true` permite abrir la app desde otro dispositivo de la red local (útil para probar en tablet/móvil).

Nota. La documentación previa menciona `localhost:5173`; este proyecto usa **5174** por configuración explícita.

5.2 Verificar que la API responde

1. Abrió la app y navegá a `/login`.

2. Inició sesión con un usuario válido.
3. Si aparece un error de red, verificá `VITE_API_URL` y que el backend esté disponible.

5.3 Previsualizar el build

```
npm run build
npm run preview
```

6. Build de producción

```
npm run build
```

- Genera la carpeta `dist/` con los assets optimizados.
- `base` es `/` (salvo en modo `capacitor`, que usa `./`).

Verificá el resultado localmente con `npm run preview` antes de desplegar.

7. Despliegue web (Vercel)

7.1 Primera puesta en marcha

1. Ingresá a Vercel y **creá un nuevo proyecto** importando el repositorio.
2. Framework preset: **Vite**.
3. Build command: `npm run build`.
4. Output directory: `dist`.
5. En **Settings** → **Environment Variables**, cargá (para Production y Preview):

Variable	Valor producción
<code>VITE_API_URL</code>	<code>https://sporttrack-sigdef.onrender.com/api</code>
<code>VITE_SIGNALR_HUB_URL</code>	<code>https://sporttrack-sigdef.onrender.com/hubs/timing</code>
<code>VITE_PUBLIC_APP_URL</code>	URL pública del sitio en Vercel

6. Deploy.

7.2 Consideraciones

- **Rutas del SPA:** Vercel sirve `index.html` para rutas desconocidas (rewrite). Verificá que `/resultados/:id` y `/planes/:id` funcionen al recargar.
- **CORS y cookies:** al ser cross-origin (Vercel → Render), la app usa **Bearer token**; las cookies third-party pueden bloquearse (esperado, ver documento 05 §11).
- **SignalR sobre HTTPS:** Vercel sirve por HTTPS; el hub debe ser accesible por WSS.

7.3 Actualizaciones

- Cada push a la rama de producción dispara un nuevo deploy.
- Los cambios de variables de entorno requieren **redesploy** para tomar efecto.

8. Build Android con Capacitor

8.1 Requisitos

- Java JDK 17 y Android Studio instalados.
- Proyecto Android sincronizado (`android/`).

8.2 Generar el build

```
# Build en modo capacitor + sincronización con Android
npm run build:android

# Abrir Android Studio
npm run open:android
```

8.3 Configuración

capacitor.config.json define:

Clave	Valor
appId	com.sporttrack.jueces
appName	SportTrack Jueces
webDir	dist
androidScheme	https
allowNavigation	https://sporttrack-sigdef.onrender.com/*
CapacitorHttp	enabled: true
Keyboard	resize: native, style: DARK
StatusBar	overlaysWebView: false, style: DARK

8.4 Notas específicas de Android

- En nativo, `api.js` usa **solo Bearer** (sin `withCredentials`) porque las cookies cross-origin rompen el login por CORS.
- El timeout de la API se amplía a **45 s** en nativo.
- Verificá que el dispositivo tenga acceso HTTPS al backend.

8.5 Generar el APK/AAB

- Con Android Studio abierto, usá **Build** → **Generate Signed Bundle / APK**.
- Firmá con tu keystore.
- Distribuí el artefacto (instalación directa o Play Store).

9. Verificación y checklist de entrega

9.1 Verificación funcional mínima

#	Verificación	Resultado esperado
1	<code>npm run dev</code> levanta en 5174	App carga sin errores en consola.
2	Login de Admin	Redirige a <code>/super</code> .
3	Login de Club	Redirige a <code>/club</code> .
4	Login de Largador/Cronometrista	Redirige a <code>/jueces/largador</code> o <code>/jueces/llegada</code> (con plan tier L).
5	Ruta pública <code>/resultados/:id</code>	Muestra resultados sin login.
6	Guard de plan	Un plan sin controles live muestra la pantalla de bloqueo.
7	Conexión SignalR	Estado "Conectado" en la consola de juez.
8	Largada de una serie	El cronometrista recibe la señal y suena el timbre.
9	Captura de tiempos	Los tiempos se reflejan en la pizarra.
10	Exportación PDF	Se descarga el archivo correcto.
11	Corte de red simulado	Los tiempos se encolan y se sincronizan al volver.
12	Colas temporales (Soporte)	Se listan, confirman y descartan entradas.

9.2 Checklist de entrega

- ☐ Node 18+ y dependencias instaladas (`npm install` sin errores).
- ☐ `.env` configurado con `VITE_API_URL`, `VITE_SIGNALR_HUB_URL`, `VITE_PUBLIC_APP_URL`.

- ☐ `npm run build` finaliza sin errores.
- ☐ `dist/` generado y verificado con `npm run preview`.
- ☐ Variables de entorno cargadas en Vercel.
- ☐ Deploy web accesible y rutas del SPA funcionando al recargar.
- ☐ Login y guards por rol/plan probados.
- ☐ SignalR conectando en producción (WSS).
- ☐ Exportaciones PDF/CSV operativas.
- ☐ Cola offline y outbox verificados con Soporte.
- ☐ (Opcional) `npm run build:android` exitoso y APK generado.
- ☐ Documentación de entrega (docs/Entrega/) revisada y actualizada.

10. Resolución de problemas de instalación

Problema	Causa	Solución
<code>npm install</code> falla	Node desactualizado o caché corrupta	Actualizar Node; <code>npm cache clean --force</code> .
El puerto 5174 está en uso	Otro proceso ocupa el puerto	Cerrar el proceso o cambiar <code>server.port</code> ; recordá <code>strictPort</code> .
Error de CORS en dev	Proxy mal configurado	Verificar <code>VITE_API_TARGET</code> y que <code>/api</code> y <code>/hubs</code> usen <code>changeOrigin</code> .
Login falla en Android	Cookies cross-origin	Ya mitigado: en nativo se usa solo Bearer. Verificar <code>CapacitorHttp</code> .
SignalR no conecta en prod	Hub/URL incorrectos	Verificar <code>VITE_SIGNALR_HUB_URL</code> y el redeploy tras cambiarla.
Rutas 404 al recargar en Vercel	Falta rewrite del SPA	Configurar fallback a <code>index.html</code> .
Build <code>capacitor</code> con assets rotos	<code>base</code> incorrecto	El modo <code>capacitor</code> debe usar <code>base: './'</code> (ya configurado).
El timbre no suena en móvil	Autoplay bloqueado	Requerir un gesto del usuario antes de la largada.

Pie del pack

Documento anterior	Documento actual	Documento siguiente
← 05 Manual Técnico	06 — Manual de Instalación y Despliegue	<i>(fin del pack)</i>

[README](#) · [01](#) · [02](#) · [03](#) · [04](#) · [05](#) · [06](#)